# HashiCorp Vault — Lab Guide, Day 1 & Day 2

Oct 16, 2026 · @Vivek Arora

## How to use this lab guide

This guide holds every hands-on step for Days 1 and 2. The study guides explain the concepts and point here with "Do Lab x.y"; each lab names the module it practises, so the two always line up.

**Day 3** continues in the separate Day 3 Lab Guide — Labs 3.1–3.10: PKI, Transit, auto-unseal, Vault Agent, Raft HA and snapshots, monitoring and audit, troubleshooting, Terraform and Ansible, Vault on Kubernetes, and the capstone. It uses the same machine and conventions; its *One-time machine setup* adds the few extra tools Day 3 needs. Expected-output images in all guides come from Vault 2.1.x.

### Lab map

| Lab | Title | Time | Study guide module | Environment |
|---|---|---|---|---|
| 1.1 | Secret-sprawl audit | 10 min | Day 1 · Module 1 | Any shell |
| 1.2 | Install Vault and run a dev server | 15 min | Day 1 · Module 3 | Dev server |
| 1.3 | Production-style server: init, unseal, seal | 35 min | Day 1 · Modules 2 & 3 | systemd server |
| 1.4 | Shamir's Secret Sharing in Python | 10 min | Day 1 · Module 2 | Python |
| 1.5 | CLI, API and UI: same task three ways | 15 min | Day 1 · Module 4 | Dev server |
| 1.6 | Tokens, policies and Userpass | 25 min | Day 1 · Module 5 | Dev server |
| 1.7 | LDAP auth with OpenLDAP | 25 min | Day 1 · Module 5 | Dev server + Docker |
| 1.8 | KV v1 vs v2 and versioning | 25 min | Day 1 · Module 6 | Dev server |
| 1.9 | Day 1 capstone | 45 min | Day 1 · all modules | Scripted Raft server |
| 2.1 | PostgreSQL dynamic secrets | 35 min | Day 2 · Module 1 | Dev server + Docker |
| 2.2 | AWS and Azure dynamic secrets (optional) | 20 min | Day 2 · Module 1 | Dev server + cloud account |
| 2.3 | Least-privilege policies and identity groups | 25 min | Day 2 · Module 2 | Dev server |
| 2.4 | AppRole and Vault Agent | 30 min | Day 2 · Module 3 | Dev server + Lab 2.1 |
| 2.5 | Kubernetes auth with kind | 35 min | Day 2 · Module 3 | kind cluster |
| 2.6 | JWT (GitHub Actions) and OIDC (optional) | 25 min | Day 2 · Module 3 | Public Vault URL / IdP |
| 2.7 | Leases, renewal and revocation | 20 min | Day 2 · Module 4 | Dev server + Lab 2.1 |
| 2.8 | Response wrapping and cubbyhole | 20 min | Day 2 · Module 5 | Dev server |
| 2.9 | Day 2 capstone | 45 min | Day 2 · all modules | Dev server + Docker |

### Lab environments

| Name | How to start it | Used by |
|---|---|---|
| Dev server | `vault server -dev -dev-root-token-id=root` in its own terminal, then `export VAULT_ADDR=http://127.0.0.1:8200 VAULT_TOKEN=root` | 1.2, 1.5–1.8, all of Day 2 except 2.5 |
| systemd server | Built in Lab 1.3 under `/etc/vault.d`, data in `/opt/vault/data` | 1.3 |
| Scripted Raft server | `day1-lab.sh` in Lab 1.9, data in `~/vault-lab` | 1.9 |
| kind cluster | Built in Lab 2.5 | 2.5 |

All three Vault servers listen on port 8200, so **only one may run at a time**: press Ctrl+C on the dev server, or `sudo systemctl stop vault`, before starting another. A dev server forgets everything when it stops, so Day 2 labs that build on Lab 2.1 should run in one dev-server session.

## One-time machine setup (Ubuntu 22.04/24.04)

```
# Tools used across the labs
$ sudo apt update
$ sudo apt install -y curl jq git unzip ldap-utils postgresql-client python3-pip docker.io
$ sudo usermod -aG docker $USER && newgrp docker      # run docker without sudo
$ pip install --user hvac psycopg2-binary

# Vault itself is installed in Lab 1.2

# Day 2, Lab 2.5 only: kubectl, helm and kind
$ sudo snap install kubectl --classic
$ curl -fsSL https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 | bash
# kind: install the release binary as described at https://kind.sigs.k8s.io (Quick Start)
```

## Conventions

- `$` lines are typed at the shell; `/ $` lines are typed inside a container or pod.
- **File blocks** start with a comment naming the file (for example `# admin.hcl`); create the file with exactly that content.
- Every **Expected output** image shows what a correct run prints. Tokens, keys, IDs, timestamps and version numbers will differ on your machine; the highlighted (amber) lines are the ones to check.
- ✓ **Checkpoint** lines tell you what must be true before moving on.
- All passwords, tokens and keys in this guide are lab-only examples.

# Lab 1.1 — Secret-sprawl audit

**Objective:** find plain-text secrets in a repository with a quick grep and a purpose-built scanner. **Time:** 10 min. **Study guide:** Day 1, Module 1. **Environment:** any shell with Docker.

## Steps

**Step 1 — Create a sample repository with planted secrets.**

```
$ mkdir -p ~/demo-repo/config ~/demo-repo/terraform && cd ~/demo-repo && git init -q

$ cat > config/application.yml <<'EOF'
spring:
  datasource:
    url: jdbc:postgresql://db:5432/app
    db_password: Pa55w0rd!
EOF

$ cat > terraform/main.tf <<'EOF'
provider "aws" {
  access_key = "AKIAEXAMPLE7QZ3W4N5V"
  secret_key = "wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY"
}
EOF

$ echo 'API_KEY=sk_demo_51HxQ2demoKEY9fT' > .env
$ git add -A && git -c user.name=trainee -c user.email=trainee@lab.local commit -qm "initial"
```

**Step 2 — Search with grep.**

```
$ grep -rniE "(password|passwd|secret[_a-z]*|api[_-]?key|token)\s*[:=]" \
    --include=*.yml --include=*.env --include=*.tf .
```

Lab 1.1 · Step 2 · grep finds plain-text secrets — EXPECTED OUTPUT

```
trainee@vault-lab:~$ cd ~/demo-repo
trainee@vault-lab:~$ grep -rniE "(password|passwd|secret[_a-z]*|api[_-]?key|token)\s*[:=]"
                     --include=*.yml --include=*.env --include=*.tf .
./config/application.yml:4:    db_password: Pa55w0rd!
./terraform/main.tf:3:  secret_key = "wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY"
./.env:1:API_KEY=sk_demo_51HxQ2demoKEY9fT
```

*Expected output — grep finds three plain-text secrets*

✓ **Checkpoint:** three hits, one per file.

**Step 3 — Scan the full Git history with gitleaks.**

```
$ docker run --rm -v "$PWD:/repo" zricethezav/gitleaks:latest detect --source=/repo -v
```

Lab 1.1 · Step 3 · gitleaks scans the Git history — EXPECTED OUTPUT

```
trainee@vault-lab:~$ docker run --rm -v "$PWD:/repo" zricethezav/gitleaks:latest detect
                        --source=/repo -v
Finding:     secret_key = "wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY"
Secret:      wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY
RuleID:      generic-api-key
File:        terraform/main.tf
Line:        3
Commit:      9c4e1b7a2f0d6e3b8a1c5f7d9e2b4a6c8d0e1f3a

... 2 more findings ...

INF 1 commits scanned.
INF scan completed in 38.6ms
WRN leaks found: 3
```

*Expected output — gitleaks reports the findings and exits with a warning*

The number of findings depends on the gitleaks rule-set version; what matters is that the scanner flags secrets that now live in Git history, even after the files are deleted.

**Step 4 — Discuss.** For each finding: who can read it, how would you rotate it, and how would you know if it leaked? Keep the answers — Labs 1.8 and 2.1 replace each one with a Vault-managed secret.

## Clean up

```
$ cd ~ && rm -rf ~/demo-repo
```

# Lab 1.2 — Install Vault and run a dev server

**Objective:** install the Vault binary, start a dev server, and store a first secret. **Time:** 15 min. **Study guide:** Day 1, Module 3. **Environment:** dev server.

## Steps

**Step 1 — Install from the HashiCorp apt repository.**

```
$ wget -O- https://apt.releases.hashicorp.com/gpg | \
    sudo gpg --dearmor -o /usr/share/keyrings/hashicorp-archive-keyring.gpg
$ echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/hashicorp-
archive-keyring.gpg] \
    https://apt.releases.hashicorp.com $(lsb_release -cs) main" | \
    sudo tee /etc/apt/sources.list.d/hashicorp.list
$ sudo apt update && sudo apt install -y vault
```

RHEL/Rocky and macOS commands are in the study guide, Day 1 Module 3.

**Step 2 — Verify, and enable tab completion.**

```
$ vault version
$ vault -h | head -12
$ vault -autocomplete-install && exec $SHELL
```

Lab 1.2 · Step 2 · Verify the installation — EXPECTED OUTPUT

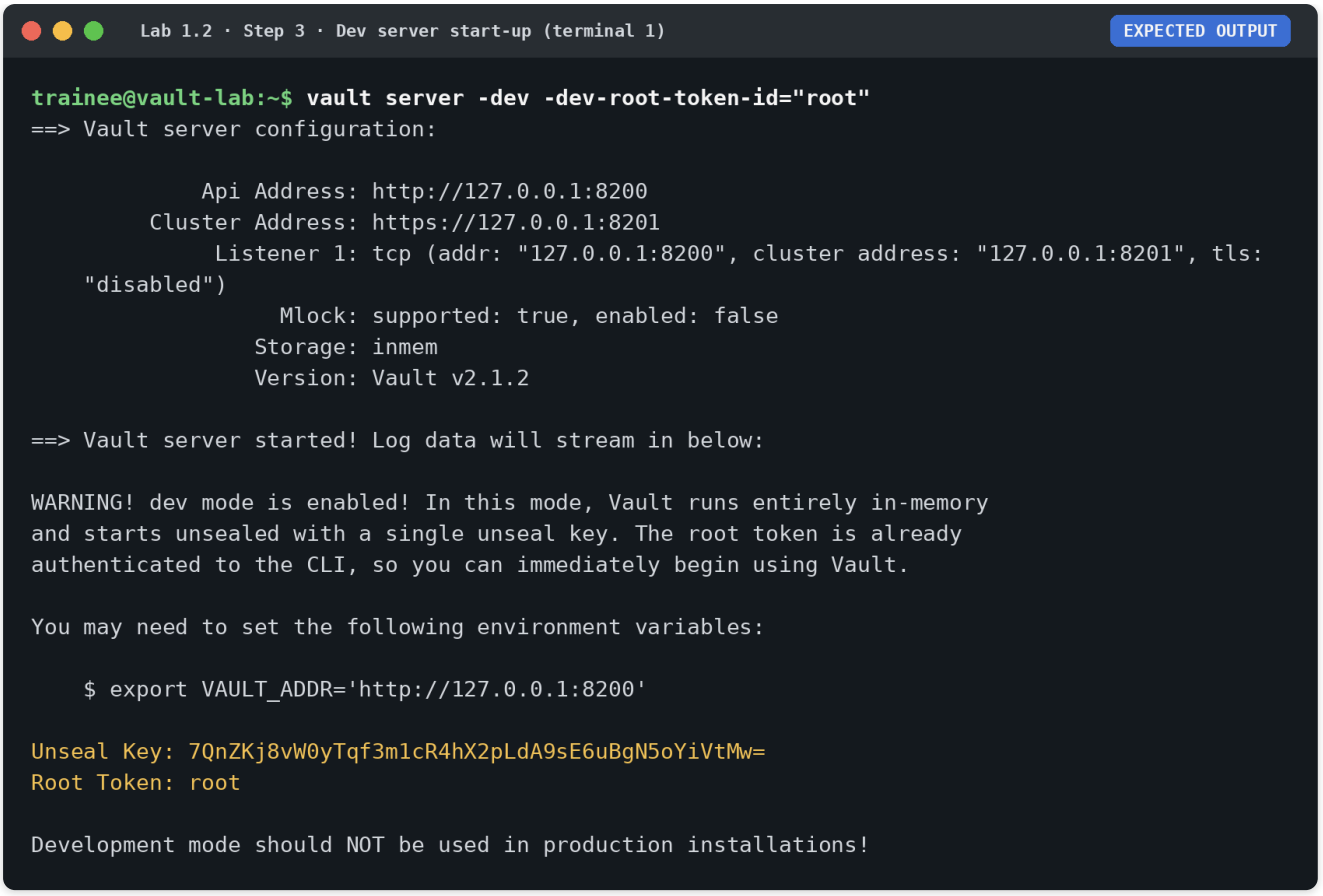
```
trainee@vault-lab:~$ vault version
Vault v2.1.2 (4c8a1f2e0b9d), built 2026-09-24T10:12:40Z
trainee@vault-lab:~$ vault -h | head -12
Usage: vault <command> [args]

Common commands:
    read        Read data and retrieves secrets
    write       Write data, configuration, and secrets
    delete      Delete secrets and configuration
    list        List data or secrets
    login       Authenticate locally
    agent       Start a Vault agent
    server      Start a Vault server
    status      Print seal and HA status
```

*Expected output — version and the common commands*

**Step 3 — Start a dev server in terminal 1.** Leave it running.

```
$ vault server -dev -dev-root-token-id="root"
```



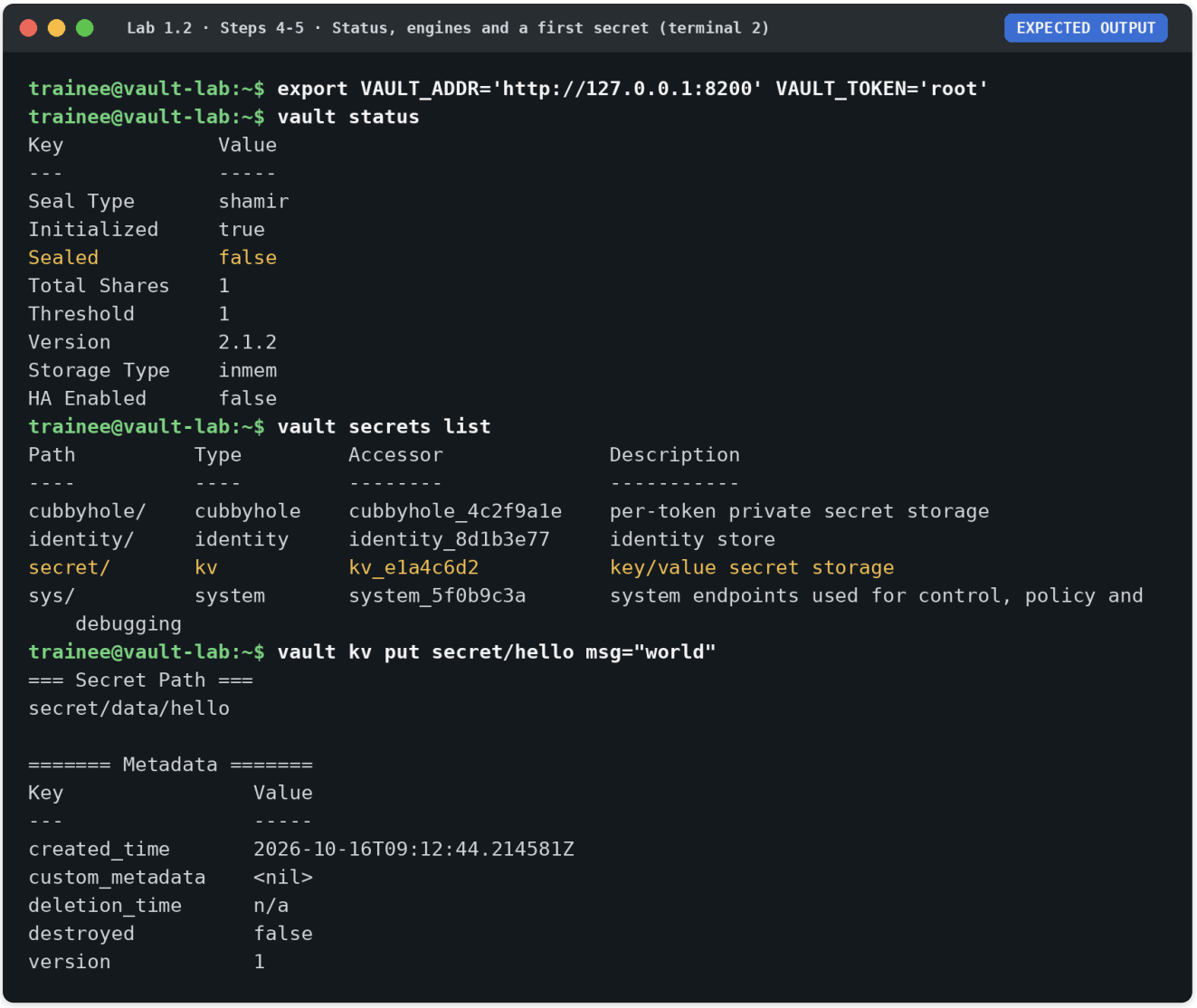

*Expected output — dev server banner with the unseal key and root token*

### Step 4 — Point the CLI at it from terminal 2 and check status.

```
$ export VAULT_ADDR='http://127.0.0.1:8200' VAULT_TOKEN='root'
$ vault status
$ vault secrets list
```

### Step 5 — Write and read a first secret.

```
$ vault kv put secret/hello msg="world"
$ vault kv get secret/hello
```




*Expected output — unsealed, secret/ pre-mounted, version 1 written*

✓ **Checkpoint:** `Sealed false`, `secret/` listed as `kv`, and `vault kv get secret/hello` shows `msg world`.

**Step 6 — Look at it in the UI.** Open `http://127.0.0.1:8200/ui`, choose method **Token**, enter `root`, open **secret/** and click **hello**. Open **Version history** to see version 1.

**Optional — the same with Docker:**

```
$ docker run -d --name vault-dev --cap-add=IPC_LOCK -p 8200:8200 \
    -e VAULT_DEV_ROOT_TOKEN_ID=root -e VAULT_DEV_LISTEN_ADDRESS=0.0.0.0:8200 \
    hashicorp/vault:latest
$ docker logs vault-dev | grep -E "Unseal Key|Root Token"
```

## Clean up

Press Ctrl+C in terminal 1. A dev server keeps everything in memory, so all data is gone; that is expected. Run `docker rm -f vault-dev` if you used Docker.

# Lab 1.3 — Production-style server: config, init, unseal, seal

**Objective:** run Vault as a systemd service with Raft storage, initialise it with Shamir key shares, unseal it, secure it, and revoke the root token. **Time:** 35 min. **Study guide:** Day 1, Modules 2 and 3. **Environment:** systemd server (stop any dev server first).

## Steps

**Step 1 — Stop the dev server** (Ctrl+C in its terminal) and clear its variables: `unset VAULT_TOKEN`.

**Step 2 — Write the configuration.** This lab disables TLS to keep the focus on Vault; the study guide shows the TLS listener.

```
$ sudo tee /etc/vault.d/vault.hcl >/dev/null <<'EOF'
ui            = true
disable_mlock = true
api_addr      = "http://127.0.0.1:8200"
cluster_addr  = "https://127.0.0.1:8201"
log_level     = "info"

storage "raft" {
  path    = "/opt/vault/data"
  node_id = "vault-1"
}

listener "tcp" {
  address     = "0.0.0.0:8200"
  tls_disable = "true"
}
EOF
```

**Step 3 — Start the service and watch the log.**

```
$ sudo systemctl enable --now vault
$ systemctl status vault --no-pager | head -5
$ journalctl -u vault -n 20 --no-pager
```

✓ **Checkpoint:** status shows `active (running)`; the log ends with a line saying Vault is sealed and waiting to be initialised or unsealed.

**Step 4 — Check the state.**

```
$ export VAULT_ADDR='http://127.0.0.1:8200'
$ vault status; echo "exit code: $?"
```

Lab 1.3 · Step 4 · A fresh server is uninitialised and sealed — EXPECTED OUTPUT

```
trainee@vault-lab:~$ export VAULT_ADDR='http://127.0.0.1:8200'
trainee@vault-lab:~$ vault status; echo "exit code: $?"
Key                Value
---                -----
Seal Type          shamir
Initialized        false
Sealed             true
Total Shares       0
Threshold          0
Unseal Progress    0/0
Unseal Nonce       n/a
Version            2.1.2
Storage Type       raft
HA Enabled         true
exit code: 2
```

*Expected output — not initialised, sealed, exit code 2*

**Step 5 — Initialise with 5 shares and a threshold of 3.**

```
$ mkdir -p ~/vault-lab
$ vault operator init -key-shares=5 -key-threshold=3 | tee ~/vault-lab/init.txt
$ chmod 600 ~/vault-lab/init.txt
```

Lab 1.3 · Step 5 · vault operator init — EXPECTED OUTPUT

```
trainee@vault-lab:~$ vault operator init -key-shares=5 -key-threshold=3 | tee
                       ~/vault-lab/init.txt
Unseal Key 1: f0WzT3mQx1uJk9Yp4cR8vA2nL6dH5sE7gB0oKiUyMtqX
Unseal Key 2: Bx9pL2vN7qW4mT1kR8cJ5yA3hD6sF0gE9uKoZiXwMbVn
Unseal Key 3: Kp3rT8wQ1mV6nY4jL9cA2xH7sD5fG0eB8uZoKiMvNbWq
Unseal Key 4: Yt6vM1qR9pW3nL8kJ2cA7xH4sD0fG5eB1uZoKiNbVmXw
Unseal Key 5: Rq2mT7vN4pW9kL1jY6cA3xH8sD2fG7eB4uZoKiVbMnQw

Initial Root Token: hvs.7mQ2vLx9TpR4nW8kY1cJ3aH6

Vault initialized with 5 key shares and a key threshold of 3. Please securely
distribute the key shares printed above. When the Vault is re-sealed,
restarted, or stopped, you must supply at least 3 of these keys to unseal it
before it can start servicing requests.

Vault does not store the generated root key. Without at least 3 keys to
reconstruct the root key, Vault will remain permanently sealed!
```

*Expected output — five unseal keys and the initial root token*

In real life each key goes to a different person (ideally encrypted with `-pgp-keys`). Saving them to a file is for the lab only.

**Step 6 — Unseal with any three keys.** Use keys 1, 3 and 5 to prove any three work.

```
$ vault operator unseal <unseal-key-1> | grep -E "Sealed|Progress"
$ vault operator unseal <unseal-key-3> | grep -E "Sealed|Progress"
$ vault operator unseal <unseal-key-5>
```

Lab 1.3 · Step 6 · Unseal with 3 of 5 keys — EXPECTED OUTPUT

```
trainee@vault-lab:~$ vault operator unseal f0WzT3mQx1uJk9Yp4cR8vA2nL6dH5sE7gB0oKiUyMtqX | grep
                       -E "Sealed|Progress"
Sealed             true
Unseal Progress    1/3
trainee@vault-lab:~$ vault operator unseal Kp3rT8wQ1mV6nY4jL9cA2xH7sD5fG0eB8uZoKiMvNbWq | grep
                       -E "Sealed|Progress"
Sealed             true
Unseal Progress    2/3
trainee@vault-lab:~$ vault operator unseal Rq2mT7vN4pW9kL1jY6cA3xH8sD2fG7eB4uZoKiVbMnQw
Key                     Value
---                     -----
Seal Type               shamir
Initialized             true
Sealed                  false
Total Shares            5
Threshold               3
Version                 2.1.2
Storage Type            raft
Cluster Name            vault-cluster-5b1e7d2c
HA Enabled              true
HA Mode                 active
Raft Committed Index    32
Raft Applied Index      32
```

*Expected output — progress 1/3, 2/3, then unsealed and active*

### Step 7 — Create the admin policy file.

```
# admin.hcl — lab administrator (tighten for production)
path "sys/*" {
  capabilities = ["create", "read", "update", "delete", "list", "sudo"]
}
path "auth/*" {
  capabilities = ["create", "read", "update", "delete", "list", "sudo"]
}
path "secret/*" {
  capabilities = ["create", "read", "update", "patch", "delete", "list"]
}
path "identity/*" {
  capabilities = ["create", "read", "update", "delete", "list"]
}
```

### Step 8 — As root: enable auditing, load the policy, create an admin user.

```
$ export VAULT_TOKEN=<initial-root-token>
$ sudo mkdir -p /var/log/vault && sudo chown vault:vault /var/log/vault
$ vault audit enable file file_path=/var/log/vault/audit.log
$ vault policy write admin admin.hcl
$ vault auth enable userpass
$ vault write auth/userpass/users/vaultadmin password='ChangeMe!123' token_policies=admin
```

### Step 9 — Switch to the admin user, then revoke root.

```
$ vault login -method=userpass username=vaultadmin
$ unset VAULT_TOKEN
```

```
$ vault token revoke <initial-root-token>
$ vault token lookup <initial-root-token>
```

Lab 1.3 · Steps 7-9 · Audit, admin login, revoke root — EXPECTED OUTPUT

```
trainee@vault-lab:~$ export VAULT_TOKEN=hvs.7mQ2vLx9TpR4nW8kY1cJ3aH6
trainee@vault-lab:~$ vault audit enable file file_path=/var/log/vault/audit.log
Success! Enabled the file audit device at: file/
trainee@vault-lab:~$ vault policy write admin admin.hcl
Success! Uploaded policy: admin
trainee@vault-lab:~$ vault auth enable userpass
Success! Enabled userpass auth method at: userpass/
trainee@vault-lab:~$ vault write auth/userpass/users/vaultadmin password='ChangeMe!123'
                       token_policies=admin
Success! Data written to: auth/userpass/users/vaultadmin
trainee@vault-lab:~$ vault login -method=userpass username=vaultadmin
Password (will be hidden):
Success! You are now authenticated.
(token details omitted)
trainee@vault-lab:~$ unset VAULT_TOKEN
From here on the CLI uses the vaultadmin token saved by vault login.
trainee@vault-lab:~$ vault token revoke hvs.7mQ2vLx9TpR4nW8kY1cJ3aH6
Success! Revoked token (if it existed)
trainee@vault-lab:~$ vault token lookup hvs.7mQ2vLx9TpR4nW8kY1cJ3aH6
Error looking up token: Error making API request.

URL: POST http://127.0.0.1:8200/v1/auth/token/lookup
Code: 403. Errors:

* bad token
```

*Expected output — audit enabled, admin login, root token revoked*

✓ **Checkpoint:** looking up the old root token returns `bad token`, and `vault token lookup` (your own token) shows policy `admin`. If you ever need root again: `vault operator generate-root` with a quorum of unseal keys.

**Step 10 — Seal, prove it is locked, unseal again.**

```
$ vault operator seal
$ vault status | grep -E "Sealed|Progress"
$ vault kv list secret/
```

Lab 1.3 · Step 10 · Seal and check — EXPECTED OUTPUT

```
trainee@vault-lab:~$ vault operator seal
Success! Vault is sealed.
trainee@vault-lab:~$ vault status | grep -E "Sealed|Progress"
Sealed             true
Unseal Progress    0/3
trainee@vault-lab:~$ vault kv list secret/
Error listing secret/: Error making API request.

URL: GET http://127.0.0.1:8200/v1/secret?list=true
Code: 503. Errors:

* Vault is sealed
```

*Expected output — sealed; every request now returns 503*

Unseal again with any three keys (Step 6). Optional: `sudo systemctl restart vault` and confirm it comes back sealed.

**Step 11 — Read the audit log.**

```
$ sudo tail -n 2 /var/log/vault/audit.log | jq -c '{type, path: .request.path, user:
.auth.display_name}'
```

## Clean up

Leave the server installed but stop it before the next lab: `sudo systemctl stop vault`. To start over completely: `sudo systemctl stop vault && sudo rm -rf /opt/vault/data/* && rm ~/vault-lab/init.txt`.

# Lab 1.4 — Shamir's Secret Sharing in Python

**Objective:** see why any 3 of 5 key shares rebuild the unseal key and 2 reveal nothing. **Time:** 10 min. **Study guide:** Day 1, Module 2. **Environment:** Python 3.8 or later; no Vault needed.

## Steps

**Step 1 — Create the script.** (Teaching aid only — Vault uses its own implementation over GF(2^8).)

```
# shamir_demo.py — split a secret into 5 shares, rebuild it from any 3
import secrets

PRIME = 2**127 - 1  # a Mersenne prime; all maths is modulo this

def split(secret: int, n: int, k: int):
    coeffs = [secret] + [secrets.randbelow(PRIME) for _ in range(k - 1)]
    def f(x):
        return sum(c * pow(x, i, PRIME) for i, c in enumerate(coeffs)) % PRIME
    return [(x, f(x)) for x in range(1, n + 1)]

def combine(shares):
    total = 0
    for j, (xj, yj) in enumerate(shares):
        num, den = 1, 1
        for m, (xm, _) in enumerate(shares):
            if m != j:
                num = (num * -xm) % PRIME
                den = (den * (xj - xm)) % PRIME
        total = (total + yj * num * pow(den, -1, PRIME)) % PRIME
    return total

secret = int.from_bytes(b"root-key-demo", "big")
shares = split(secret, n=5, k=3)
for s in shares:
    print("share", s[0], hex(s[1]))

print(combine(shares[:3]) == secret)                          # shares 1, 2, 3
print(combine([shares[0], shares[4], shares[2]]) == secret)  # shares 1, 5, 3
print(combine(shares[:2]) == secret)                          # only 2 shares
```

**Step 2 — Run it.**

```
$ python3 shamir_demo.py
```

Lab 1.4 · Step 2 · Shamir split and combine — EXPECTED OUTPUT

```
trainee@vault-lab:~$ python3 shamir_demo.py
share 1 0x2f9c41d7a63e0b85e1c4f72a9d0b36e5
share 2 0x6a1e93c0d85f27b4c3a90e6f1d72b8a4
share 3 0x1c7b05e9f24a8d36b0e5c19a7f43d062
share 4 0x5d38e7a2c09b41f6a7d2e04b93c5f1a8
share 5 0x3e90b4c6f17d52a8e9c3b06d2a4f7e19
True
True
False
```

*Expected output — five shares, then True, True, False*

Share values are random on every run; the three booleans are always `True`, `True`, `False`.

**Step 3 — Experiment.**

1. Change to `split(secret, n=7, k=4)` and confirm 4 shares work and 3 do not.
2. Print `combine(shares[:2])` itself: the result is a random-looking number unrelated to the secret — two shares leak nothing.
3. Relate it back to Lab 1.3: `-key-shares` is `n`, `-key-threshold` is `k`.

## Clean up

`rm shamir_demo.py` (optional).

# Lab 1.5 — CLI, API and UI: the same task three ways

**Objective:** prove the CLI and UI are clients of the HTTP API by doing one task through all three. **Time:** 15 min. **Study guide:** Day 1, Module 4. **Environment:** dev server.

## Setup

```
$ sudo systemctl stop vault 2>/dev/null; vault server -dev -dev-root-token-id=root   #
terminal 1
$ export VAULT_ADDR=http://127.0.0.1:8200 VAULT_TOKEN=root                            #
terminal 2
```

## Part A — The CLI

```
$ vault kv put secret/ex4 team=payments
$ vault kv get -field=team secret/ex4
$ vault kv get -output-curl-string secret/ex4
```

Lab 1.5 · Part A · The CLI — EXPECTED OUTPUT

```
trainee@vault-lab:~$ vault kv put secret/ex4 team=payments
=== Secret Path ===
secret/data/ex4

======= Metadata =======
Key                Value
---                -----
created_time       2026-10-16T10:04:19.882034Z
deletion_time      n/a
destroyed          false
version            1
trainee@vault-lab:~$ vault kv get -field=team secret/ex4
payments
trainee@vault-lab:~$ vault kv get -output-curl-string secret/ex4
curl -H "X-Vault-Request: true" -H "X-Vault-Token: $(vault print token)"
    http://127.0.0.1:8200/v1/secret/data/ex4
```

*Expected output — version 1 written, field read, and the equivalent curl command*

`-output-curl-string` prints the API call instead of running it: this is the bridge to Part B.

## Part B — The HTTP API

```
$ curl -s -H "X-Vault-Token: $VAULT_TOKEN" -X POST \
    -d '{"data":{"team":"payments","owner":"vivek"}}' \
    $VAULT_ADDR/v1/secret/data/ex4 | jq .data
$ curl -s -H "X-Vault-Token: $VAULT_TOKEN" $VAULT_ADDR/v1/secret/data/ex4 | jq .data.data
$ curl -s $VAULT_ADDR/v1/sys/health | jq '{initialized, sealed, standby}'
```

Lab 1.5 · Part B · The HTTP API — EXPECTED OUTPUT

```
trainee@vault-lab:~$ curl -s -H "X-Vault-Token: $VAULT_TOKEN" -X POST -d
                       '{"data":{"team":"payments","owner":"vivek"}}'
                       $VAULT_ADDR/v1/secret/data/ex4 | jq .data
{
  "created_time": "2026-10-16T10:06:51.140337Z",
  "custom_metadata": null,
  "deletion_time": "",
  "destroyed": false,
  "version": 2
}
trainee@vault-lab:~$ curl -s -H "X-Vault-Token: $VAULT_TOKEN" $VAULT_ADDR/v1/secret/data/ex4 |
                       jq .data.data
{
  "owner": "vivek",
  "team": "payments"
}
trainee@vault-lab:~$ curl -s $VAULT_ADDR/v1/sys/health | jq '{initialized, sealed, standby}'
{
  "initialized": true,
  "sealed": false,
  "standby": false
}
```

*Expected output — the API write creates version 2; health needs no token*

✓ **Checkpoint:** the API write returned `"version": 2` — it updated the same secret the CLI created.

## Part C — The UI

1. Open `http://127.0.0.1:8200/ui`, method **Token**, token `root`.
2. **Secrets Engines** → **secret/** → **ex4**. Confirm `owner` and `team` are shown.
3. Click **Create new version**, add `region = ap-south-1`, **Save**. Open **Version history**: three versions.
4. Click the terminal icon (**Web REPL**) and run `read secret/data/ex4` — the UI's own CLI.
5. Back in terminal 2: `vault kv get secret/ex4` shows version 3 with `region`.

## Part D — Compare

| Task | CLI | API | UI |
|---|---|---|---|
| Write `team=payments` | `vault kv put secret/ex4 team=payments` | `POST /v1/secret/data/ex4` | Create new version |
| Read it | `vault kv get secret/ex4` | `GET /v1/secret/data/ex4` | Click the secret |
| List engines | `vault secrets list` | `GET /v1/sys/mounts` | Secrets Engines page |
| Who am I | `vault token lookup` | `GET /v1/auth/token/lookup-self` | Web REPL: `read auth/token/lookup-self` |

Run each CLI command in the table with `-output-curl-string` and compare it with the API column.

## Clean up

Keep the dev server running for Lab 1.6.

# Lab 1.6 — Tokens, policies and Userpass

**Objective:** write policies, issue scoped tokens, see the token tree in action, and log in as a Userpass user. **Time:** 25 min. **Study guide:** Day 1, Module 5. **Environment:** dev server ( `VAULT_TOKEN=root` ).

## Steps

### Step 1 — Create two policy files.

```
# dev-read.hcl — read-only access to one app's secrets
path "secret/data/myapp/*" {
  capabilities = ["read"]
}
path "secret/metadata/myapp/*" {
  capabilities = ["list"]
}
```

Create `admin.hcl` exactly as in Lab 1.3, Step 7.

### Step 2 — Load them, add a secret, issue a token.

```
$ vault policy write dev-read dev-read.hcl
$ vault policy write admin admin.hcl
$ vault kv put secret/myapp/config db_user=app db_pass=S3cret
$ vault token create -policy=dev-read -ttl=1h
```

### Step 3 — Ask Vault what that token can do.

```
$ vault token capabilities <token-from-step-2> secret/data/myapp/config
```

Lab 1.6 · Steps 1-3 · Policy and a scoped token — EXPECTED OUTPUT

```
trainee@vault-lab:~$ vault policy write dev-read dev-read.hcl
Success! Uploaded policy: dev-read
trainee@vault-lab:~$ vault kv put secret/myapp/config db_user=app db_pass=S3cret
trainee@vault-lab:~$ vault token create -policy=dev-read -ttl=1h
Key                  Value
---                  -----
token                hvs.CAESIDf7kQx2m9TnWvR4pL8yB3cH
token_accessor       Z8nPq4rT1vWm6kY2jL9cA3xH
token_duration       1h
token_renewable      true
token_policies       ["default" "dev-read"]
identity_policies    []
policies             ["default" "dev-read"]
trainee@vault-lab:~$ vault token capabilities hvs.CAESIDf7kQx2m9TnWvR4pL8yB3cH
                         secret/data/myapp/config
read
```

*Expected output — token with dev-read; capability on the path is read*

### Step 4 — Use the token: read works, write is denied.

```
$ export T=<token-from-step-2>
$ VAULT_TOKEN=$T vault kv get -field=db_user secret/myapp/config
```

```
$ VAULT_TOKEN=$T vault kv put secret/myapp/config db_user=hacker
```

Lab 1.6 · Step 4 · Read allowed, write denied — EXPECTED OUTPUT

```
trainee@vault-lab:~$ export T=hvs.CAESIDf7kQx2m9TnWvR4pL8yB3cH
trainee@vault-lab:~$ VAULT_TOKEN=$T vault kv get -field=db_user secret/myapp/config
app
trainee@vault-lab:~$ VAULT_TOKEN=$T vault kv put secret/myapp/config db_user=hacker
Error writing data to secret/data/myapp/config: Error making API request.

URL: PUT http://127.0.0.1:8200/v1/secret/data/myapp/config
Code: 403. Errors:

* 1 error occurred:
	* permission denied
```

*Expected output — read succeeds, write returns 403 permission denied*

**Step 5 — Inspect, renew and revoke.**

```
$ vault token lookup $T                # policies, ttl, accessor
$ vault token renew $T                 # resets the TTL to 1h
$ vault token revoke $T
```

**Steps 6–7 — The token tree.** A parent with the `admin` policy creates a child; revoking the parent kills the child.

```
$ PARENT=$(vault token create -policy=admin -ttl=1h -field=token)
$ CHILD=$(VAULT_TOKEN=$PARENT vault token create -policy=dev-read -field=token)
$ vault token lookup -format=json $CHILD | jq -r .data.policies[]
$ vault token revoke $PARENT
$ vault token lookup $CHILD
```

Lab 1.6 · Steps 6-7 · Token tree: revoke the parent, the child dies — EXPECTED OUTPUT

```
trainee@vault-lab:~$ PARENT=$(vault token create -policy=admin -ttl=1h -field=token)
trainee@vault-lab:~$ CHILD=$(VAULT_TOKEN=$PARENT vault token create -policy=dev-read
                      -field=token)
trainee@vault-lab:~$ vault token lookup -format=json $CHILD | jq -r .data.policies[]
default
dev-read
trainee@vault-lab:~$ vault token revoke $PARENT
Success! Revoked token (if it existed)
trainee@vault-lab:~$ vault token lookup $CHILD
Error looking up token: Error making API request.

Code: 403. Errors:

* bad token
```

*Expected output — the child token is gone with its parent*

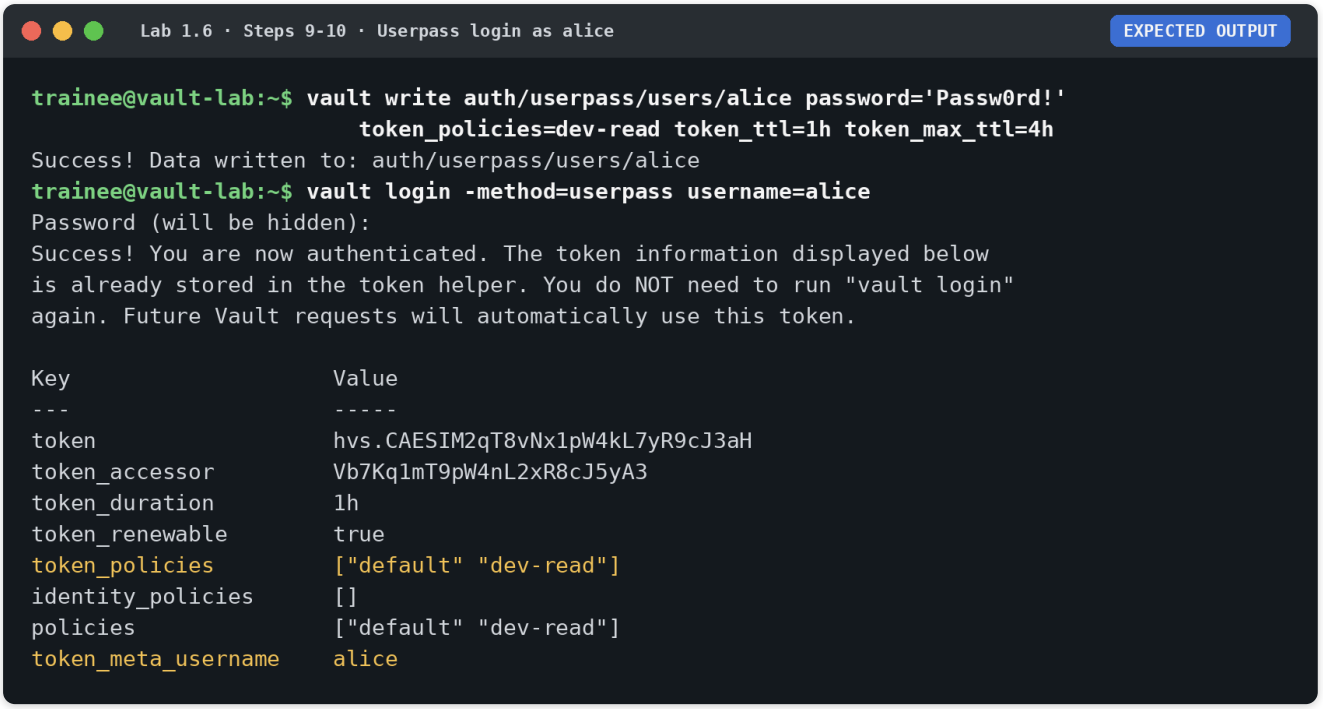
✓ **Checkpoint:** the last command fails with `bad token`. Repeat with `-orphan` on the child (needs `sudo` on `auth/token/create`, which `admin` grants through `auth/*`) and it survives.

**Step 8 — Enable Userpass and tune its TTLs.**

```
$ vault auth enable userpass
$ vault auth tune -default-lease-ttl=1h -max-lease-ttl=8h userpass/
```

**Steps 9–10 — Create alice and log in as her.**

```
$ vault write auth/userpass/users/alice password='Passw0rd!' \
    token_policies=dev-read token_ttl=1h token_max_ttl=4h
$ vault login -method=userpass username=alice        # password Passw0rd!
```




*Expected output — alice is authenticated with default and dev-read*

**Step 11 — Act as alice.** `VAULT_TOKEN=root` is still exported and would override her token, so unset it first.

```
$ unset VAULT_TOKEN
$ vault kv get secret/myapp/config        # allowed
$ vault kv put secret/myapp/config x=y    # 403
$ vault policy list                       # 403
$ export VAULT_TOKEN=root                 # back to admin for the next lab
```

✓ **Checkpoint:** alice can read `secret/myapp/config` and nothing else.

## Clean up

Keep the dev server running for Lab 1.7 (it needs `dev-read`).

# Lab 1.7 — LDAP auth with OpenLDAP

**Objective:** connect Vault to a directory and grant policies through an LDAP group. **Time:** 25 min. **Study guide:** Day 1, Module 5. **Environment:** dev server from Lab 1.6 (needs the `dev-read` policy) + Docker.

## Steps

**Step 1 — Start OpenLDAP.**

```
$ docker run -d --name openldap -p 389:389 \
    -e LDAP_ORGANISATION="StackMaster Lab" \
    -e LDAP_DOMAIN="lab.local" \
    -e LDAP_ADMIN_PASSWORD="admin" \
    osixia/openldap:1.5.0
$ sleep 10      # give slapd time to start
```

**Step 2 — Load one user ( `bob` ) in one group ( `devops` ).**

```
# lab.ldif
dn: ou=users,dc=lab,dc=local
objectClass: organizationalUnit
ou: users

dn: ou=groups,dc=lab,dc=local
objectClass: organizationalUnit
ou: groups

dn: uid=bob,ou=users,dc=lab,dc=local
objectClass: inetOrgPerson
cn: Bob Kumar
sn: Kumar
uid: bob
userPassword: BobPass123

dn: cn=devops,ou=groups,dc=lab,dc=local
objectClass: groupOfNames
cn: devops
member: uid=bob,ou=users,dc=lab,dc=local
```

```
$ ldapadd -x -H ldap://localhost:389 -D "cn=admin,dc=lab,dc=local" -w admin -f lab.ldif
$ ldapsearch -x -H ldap://localhost:389 -D "cn=admin,dc=lab,dc=local" -w admin \
    -b "dc=lab,dc=local" "(cn=devops)" member
```

Lab 1.7 · Step 2 · Load the directory — EXPECTED OUTPUT

```
trainee@vault-lab:~$ ldapadd -x -H ldap://localhost:389 -D "cn=admin,dc=lab,dc=local" -w admin
                       -f lab.ldif
adding new entry "ou=users,dc=lab,dc=local"

adding new entry "ou=groups,dc=lab,dc=local"

adding new entry "uid=bob,ou=users,dc=lab,dc=local"

adding new entry "cn=devops,ou=groups,dc=lab,dc=local"

trainee@vault-lab:~$ ldapsearch -x -H ldap://localhost:389 -D "cn=admin,dc=lab,dc=local" -w
                       admin -b "dc=lab,dc=local" "(cn=devops)" member
dn: cn=devops,ou=groups,dc=lab,dc=local
member: uid=bob,ou=users,dc=lab,dc=local
```

*Expected output — four entries added; bob is a member of devops*

### Step 3 — Enable and configure the LDAP method.

```
$ vault auth enable ldap
$ vault write auth/ldap/config \
    url="ldap://localhost:389" \
    binddn="cn=admin,dc=lab,dc=local" bindpass="admin" \
    userdn="ou=users,dc=lab,dc=local" userattr="uid" \
    groupdn="ou=groups,dc=lab,dc=local" \
    groupfilter="(&(objectClass=groupOfNames)(member={{.UserDN}}))" \
    groupattr="cn" \
    starttls=false insecure_tls=true
```

### Steps 4–5 — Map the group to a policy and log in as bob.

```
$ vault write auth/ldap/groups/devops policies=dev-read
$ vault login -method=ldap username=bob             # password BobPass123
```

Lab 1.7 · Steps 4-5 · Bob logs in through LDAP — EXPECTED OUTPUT

```
trainee@vault-lab:~$ vault write auth/ldap/groups/devops policies=dev-read
Success! Data written to: auth/ldap/groups/devops
trainee@vault-lab:~$ vault login -method=ldap username=bob
Password (will be hidden):
Success! You are now authenticated. The token information displayed below
is already stored in the token helper.

Key                    Value
---                    -----
token                  hvs.CAESIKp3vR8nT1mW6qL9yJ2cH4xA
token_accessor         Nq8Wm2kT5vR1pL7yJ3cA9xH4
token_duration         768h
token_renewable        true
token_policies         ["default" "dev-read"]
identity_policies      []
policies               ["default" "dev-read"]
token_meta_username    bob
```

*Expected output — bob receives dev-read through the devops group*

✓ **Checkpoint:** `token_policies` contains `dev-read` and `token_meta_username` is `bob`.

**Step 6 — Act as bob, then switch back.**

```
$ unset VAULT_TOKEN
$ vault kv get secret/myapp/config        # allowed via the group
$ vault kv put secret/myapp/config x=y    # 403
$ export VAULT_TOKEN=root
```

**Step 7 — Prove the group is what grants access.** Remove the mapping and log in again: bob now gets only `default`.

```
$ vault delete auth/ldap/groups/devops
$ vault login -method=ldap username=bob | grep policies
$ export VAULT_TOKEN=root
```

**Step 8 — Failure case.** `vault login -method=ldap username=bob password=wrong` must fail with an LDAP bind error — Vault never stores LDAP passwords.

## Clean up

```
$ vault auth disable ldap
$ docker rm -f openldap
```

# Lab 1.8 — KV v1 vs v2 and versioning

**Objective:** feel the difference between KV v1 and v2, and walk a secret through every KV v2 lifecycle state. **Time:** 25 min. **Study guide:** Day 1, Module 6. **Environment:** dev server ( `VAULT_TOKEN=root` ).

## Part A — KV v1: no history

```
$ vault secrets enable -path=kv1 -version=1 kv
$ vault kv put kv1/app/config user=admin pass=one
$ vault kv put kv1/app/config user=admin pass=two
$ vault kv get kv1/app/config
```

Lab 1.8 · Part A · KV v1 overwrites, no history — EXPECTED OUTPUT

```
trainee@vault-lab:~$ vault secrets enable -path=kv1 -version=1 kv
Success! Enabled the kv secrets engine at: kv1/
trainee@vault-lab:~$ vault kv put kv1/app/config user=admin pass=one
Success! Data written to: kv1/app/config
trainee@vault-lab:~$ vault kv put kv1/app/config user=admin pass=two
Success! Data written to: kv1/app/config
trainee@vault-lab:~$ vault kv get kv1/app/config
==== Data ====
Key     Value
---     -----
pass    two
user    admin
Version 1 ("pass=one") is gone for good: KV v1 keeps no history.
```

*Expected output — the second write simply replaced the first*

The API has no `data/` segment on v1:

```
$ curl -s -H "X-Vault-Token: $VAULT_TOKEN" $VAULT_ADDR/v1/kv1/app/config | jq .data
```

## Part B — KV v2: versions, soft delete, destroy, check-and-set

### Step 1 — Enable v2 and write three versions.

```
$ vault secrets enable -path=kv2 kv-v2
$ vault kv put   -mount=kv2 app/config user=admin pass=one     # version 1
$ vault kv put   -mount=kv2 app/config user=admin pass=two     # version 2
$ vault kv patch -mount=kv2 app/config pass=three              # version 3, keeps user
```

### Step 2 — Read old versions and the metadata.

```
$ vault kv get -mount=kv2 -version=1 -field=pass app/config
$ vault kv metadata get -mount=kv2 app/config | head -14
```

Lab 1.8 · Part B · Three versions and the metadata — EXPECTED OUTPUT

```
trainee@vault-lab:~$ vault kv patch -mount=kv2 app/config pass=three
=== Secret Path ===
kv2/data/app/config

======= Metadata =======
Key                Value
---                -----
created_time       2026-10-16T10:31:07.551248Z
deletion_time      n/a
destroyed          false
version            3
trainee@vault-lab:~$ vault kv get -mount=kv2 -version=1 -field=pass app/config
one
trainee@vault-lab:~$ vault kv metadata get -mount=kv2 app/config | head -14
=== Metadata Path ===
kv2/metadata/app/config

========== Metadata ==========
Key                     Value
---                     -----
cas_required            false
created_time            2026-10-16T10:30:41.208913Z
current_version         3
custom_metadata         <nil>
delete_version_after    0s
max_versions            0
oldest_version          0
updated_time            2026-10-16T10:31:07.551248Z
```

*Expected output — version 3 written; version 1 still readable; current_version 3*

### Step 3 — Soft delete, undelete, destroy.

```
$ vault kv delete   -mount=kv2 app/config                # soft-deletes the latest (v3)
$ vault kv get      -mount=kv2 app/config                # metadata only, deletion_time set
$ vault kv undelete -mount=kv2 -versions=3 app/config
$ vault kv destroy  -mount=kv2 -versions=1 app/config    # permanent
$ vault kv get -mount=kv2 -version=1 app/config | grep destroyed
```

Lab 1.8 · Part B · Delete, undelete, destroy — EXPECTED OUTPUT

```
trainee@vault-lab:~$ vault kv delete -mount=kv2 app/config
Success! Data deleted (if it existed) at: kv2/data/app/config
trainee@vault-lab:~$ vault kv get -mount=kv2 app/config
=== Secret Path ===
kv2/data/app/config

======= Metadata =======
Key                Value
---                -----
created_time       2026-10-16T10:31:07.551248Z
deletion_time      2026-10-16T10:33:52.118604Z
destroyed          false
version            3
trainee@vault-lab:~$ vault kv undelete -mount=kv2 -versions=3 app/config
Success! Data written to: kv2/undelete/app/config
trainee@vault-lab:~$ vault kv destroy -mount=kv2 -versions=1 app/config
Success! Data written to: kv2/destroy/app/config
trainee@vault-lab:~$ vault kv get -mount=kv2 -version=1 app/config | grep destroyed
destroyed          true
```

*Expected output — deletion_time set, then version 1 destroyed*

### Step 4 — Roll back, then protect writes with check-and-set.

```
$ vault kv rollback -mount=kv2 -version=2 app/config | grep version     # creates v4 = copy
of v2
$ vault kv metadata put -mount=kv2 -cas-required=true app/config
$ vault kv put -mount=kv2 -cas=3 app/config user=admin pass=four        # fails
$ vault kv put -mount=kv2 -cas=4 app/config user=admin pass=four | grep version
```

Lab 1.8 · Part B · Check-and-set protects against overwrites — EXPECTED OUTPUT

```
trainee@vault-lab:~$ vault kv rollback -mount=kv2 -version=2 app/config | grep version
version            4
trainee@vault-lab:~$ vault kv metadata put -mount=kv2 -cas-required=true app/config
Success! Data written to: kv2/metadata/app/config
trainee@vault-lab:~$ vault kv put -mount=kv2 -cas=3 app/config user=admin pass=four
Error writing data to kv2/data/app/config: Error making API request.

Code: 400. Errors:

* check-and-set parameter did not match the current version
trainee@vault-lab:~$ vault kv put -mount=kv2 -cas=4 app/config user=admin pass=four | grep
                       version
version            5
```

*Expected output — the stale -cas=3 write is rejected; -cas=4 creates version 5*

✓ **Checkpoint:** `vault kv metadata get -mount=kv2 app/config` shows `current_version 5`, `cas_required true`, and version 1 `destroyed true`.

### Step 5 — Metadata controls and safe input.

```
$ vault kv metadata put -mount=kv2 -max-versions=5 -delete-version-after=720h \
    -custom-metadata=owner=payments app/config
$ vault kv put -mount=kv2 app/db password=-          # type the value, then Ctrl+D: nothing
```

```
in shell history
$ echo '{"api_key":"abc123"}' > payload.json && vault kv put -mount=kv2 app/json
@payload.json
```

**Step 6 — Upgrade the v1 mount to v2.**

```
$ vault kv enable-versioning kv1/
$ vault kv get -mount=kv1 app/config                    # now has metadata and version 1
```

**Step 7 (optional) — From Python.** Run `kv_demo.py` from the study guide (Day 1, Module 6) against this server: `python3 kv_demo.py`.

## Clean up

```
$ vault kv metadata delete -mount=kv2 app/config
$ vault secrets disable kv1 && vault secrets disable kv2
$ rm -f payload.json
```

# Lab 1.9 — Day 1 capstone

**Objective:** build a production-style Vault from nothing in one script — Raft storage, Shamir unseal, audit, KV v2, policies, Userpass — then have a Python app read its config with least privilege. **Time:** 45 min. **Study guide:** Day 1, all modules. **Environment:** scripted Raft server in `~/vault-lab`.

| Script step | What it does | Module |
|---|---|---|
| 1–2 | Write `vault.hcl` (Raft, TCP listener) and start the server | 3 |
| 3–4 | Initialise with 5 shares / threshold 3, unseal with 3 keys | 2 |
| 5 | Enable a file audit device | 1, 3 |
| 6 | Mount KV v2 at `secret/`, write two versions of the app config | 6 |
| 7 | Write `admin` and `app-read` policies | 5 |
| 8 | Enable Userpass; create `vaultadmin` and `myapp` | 5 |
| 9 | Log in as `myapp`: read allowed, write denied | 4, 5 |
| 10 | Log in as `vaultadmin`, then revoke the root token | 2, 3 |

## Step 1 — Free port 8200

```
$ sudo systemctl stop vault 2>/dev/null     # Lab 1.3 server
# and press Ctrl+C on any dev server
$ unset VAULT_TOKEN
```

## Step 2 — Create and run the build script

```
#!/usr/bin/env bash
# day1-lab.sh — build the Day 1 environment end to end (LAB ONLY)
set -euo pipefail

export VAULT_ADDR="http://127.0.0.1:8200"
LAB="$HOME/vault-lab"
mkdir -p "$LAB/data" "$LAB/logs"
cd "$LAB"

# 1. Configuration: Raft storage, HTTP listener on localhost
cat > vault.hcl <<EOF
ui            = true
disable_mlock = true
api_addr      = "http://127.0.0.1:8200"
cluster_addr  = "https://127.0.0.1:8201"

storage "raft" {
  path    = "$LAB/data"
  node_id = "lab-node-1"
}

listener "tcp" {
  address     = "127.0.0.1:8200"
  tls_disable = "true"
}
EOF
```

```
# 2. Start the server in the background and wait for the listener
vault server -config=vault.hcl > logs/server.log 2>&1 &
echo $! > vault.pid
until curl -s -o /dev/null "$VAULT_ADDR/v1/sys/health"; do sleep 1; done

# 3. Initialise: 5 shares, threshold 3
vault operator init -key-shares=5 -key-threshold=3 -format=json > init.json
chmod 600 init.json

# 4. Unseal with 3 of the 5 keys
for i in 0 1 2; do
  vault operator unseal "$(jq -r ".unseal_keys_b64[$i]" init.json)" > /dev/null
done
vault status
export VAULT_TOKEN="$(jq -r '.root_token' init.json)"

# 5. Audit device
vault audit enable file file_path="$LAB/logs/audit.log"

# 6. KV v2 and two versions of the app config
vault secrets enable -path=secret kv-v2
vault kv put secret/myapp/config db_user=app_rw db_pass='S3cure!Pass'
api_url=https://api.lab.local
vault kv put secret/myapp/config db_user=app_rw db_pass='R0tated!Pass'
api_url=https://api.lab.local

# 7. Policies
cat > app-read.hcl <<'EOF'
path "secret/data/myapp/*" {
  capabilities = ["read"]
}
path "secret/metadata/myapp/*" {
  capabilities = ["read", "list"]
}
EOF

cat > admin.hcl <<'EOF'
path "sys/*" {
  capabilities = ["create", "read", "update", "delete", "list", "sudo"]
}
path "auth/*" {
  capabilities = ["create", "read", "update", "delete", "list", "sudo"]
}
path "secret/*" {
  capabilities = ["create", "read", "update", "patch", "delete", "list"]
}
EOF

vault policy write app-read app-read.hcl
vault policy write admin admin.hcl

# 8. Userpass identities
vault auth enable userpass
vault write auth/userpass/users/vaultadmin password='Adm1n!Lab' token_policies=admin
vault write auth/userpass/users/myapp password='App!Lab123' \
  token_policies=app-read token_ttl=30m token_max_ttl=2h

# 9. Least privilege check as the app identity
APP_TOKEN="$(vault login -method=userpass -token-only username=myapp password='App!Lab123')"
VAULT_TOKEN="$APP_TOKEN" vault kv get secret/myapp/config
if VAULT_TOKEN="$APP_TOKEN" vault kv put secret/myapp/config hack=1 2>/dev/null; then
```

```
  echo "UNEXPECTED: write was allowed"
else
  echo "OK: write denied by policy app-read"
fi

# 10. Confirm the admin works, then revoke root
ADMIN_TOKEN="$(vault login -method=userpass -token-only username=vaultadmin
password='Adm1n!Lab')"
VAULT_TOKEN="$ADMIN_TOKEN" vault secrets list
vault token revoke "$VAULT_TOKEN"
echo "Done. Root token revoked. Audit log: $LAB/logs/audit.log"
```

```
$ chmod +x day1-lab.sh && ./day1-lab.sh
```

Lab 1.9 · Step 2 · Run the Day 1 build script — EXPECTED OUTPUT

```
trainee@vault-lab:~$ ./day1-lab.sh
Key                     Value
---                     -----
Seal Type               shamir
Initialized             true
Sealed                  false
Total Shares            5
Threshold               3
Storage Type            raft
HA Mode                 active
Success! Enabled the file audit device at: file/
Success! Enabled the kv-v2 secrets engine at: secret/
Success! Uploaded policy: app-read
Success! Uploaded policy: admin
Success! Enabled userpass auth method at: userpass/
====== Data ======
Key        Value
---        -----
api_url    https://api.lab.local
db_pass    R0tated!Pass
db_user    app_rw
OK: write denied by policy app-read
Path       Type     Accessor          Description
----       ----     --------          -----------
secret/    kv       kv_7a2c9e14       n/a
(cubbyhole/, identity/ and sys/ rows omitted)
Done. Root token revoked. Audit log: /home/trainee/vault-lab/logs/audit.log
```

*Expected output — unsealed, everything enabled, write denied, root revoked*

## Step 3 — Create the application

```
# app.py — loads its configuration from Vault at startup (pip install hvac)
import os
import sys

import hvac

client = hvac.Client(url=os.getenv("VAULT_ADDR", "http://127.0.0.1:8200"))

# 1. Authenticate (Day 2 replaces Userpass with AppRole or Kubernetes auth)
login = client.auth.userpass.login(
    username=os.environ["APP_USER"],
```

```
    password=os.environ["APP_PASS"],
)
auth = login["auth"]
print(f"Logged in: policies={auth['policies']}, ttl={auth['lease_duration']}s")

# 2. Read the latest version of the config
try:
    secret = client.secrets.kv.v2.read_secret_version(
        mount_point="secret", path="myapp/config", raise_on_deleted_version=True
    )
except hvac.exceptions.Forbidden:
    sys.exit("Policy does not allow reading secret/data/myapp/config")

cfg = secret["data"]["data"]
version = secret["data"]["metadata"]["version"]
print(f"Config v{version}: db_user={cfg['db_user']} api_url={cfg['api_url']}")
db_password = cfg["db_pass"]          # use it; never log it

# 3. Prove least privilege: writes must fail
try:
    client.secrets.kv.v2.create_or_update_secret(
        mount_point="secret", path="myapp/config", secret={"x": "y"}
    )
    print("WARNING: write succeeded — policy is too broad")
except hvac.exceptions.Forbidden:
    print("Write denied, as expected")

# 4. Renew before the TTL runs out, then revoke the token on shutdown
client.auth.token.renew_self(increment="30m")
client.logout(revoke_token=True)
```

## Step 4 — Run it and read the audit trail

```
$ export VAULT_ADDR=http://127.0.0.1:8200 APP_USER=myapp APP_PASS='App!Lab123'
$ python3 app.py
$ tail -n 2 ~/vault-lab/logs/audit.log | jq -c '{type, path: .request.path, user:
.auth.display_name}'
```

Lab 1.9 · Steps 3-4 · The application and the audit log — EXPECTED OUTPUT

```
trainee@vault-lab:~$ export VAULT_ADDR=http://127.0.0.1:8200 APP_USER=myapp
                       APP_PASS='App!Lab123'
trainee@vault-lab:~$ python3 app.py
Logged in: policies=['app-read', 'default'], ttl=1800s
Config v2: db_user=app_rw api_url=https://api.lab.local
Write denied, as expected
trainee@vault-lab:~$ tail -n 2 ~/vault-lab/logs/audit.log | jq -c '{type, path: .request.path,
                       user: .auth.display_name}'
{"type":"request","path":"auth/token/revoke-self","user":"userpass-myapp"}
{"type":"response","path":"auth/token/revoke-self","user":"userpass-myapp"}
```

*Expected output — app reads config v2, write denied; audit shows the token revoke*

## Self-check

- ☐ `vault status` shows `Sealed false` and `Storage Type raft`
- ☐ `init.json` holds 5 unseal keys; any 3 unseal the server after `kill $(cat ~/vault-lab/vault.pid)` and a restart with `vault server -config=$HOME/vault-lab/vault.hcl &`

- ☐ `vault kv metadata get secret/myapp/config` shows versions 1 and 2 (log in as `vaultadmin` first)
- ☐ `myapp` can read but not write; `vaultadmin` can list secrets engines
- ☐ The initial root token returns `bad token`
- ☐ Every request appears in `audit.log`, with secret values HMAC-hashed

**Stretch goals:** add the LDAP method from Lab 1.7 and map `devops` to `app-read`; restart the server and unseal through the API with `curl`; set `-max-versions=3` and write four versions to watch the oldest disappear.

## Clean up

```
$ kill "$(cat ~/vault-lab/vault.pid)" && rm -rf ~/vault-lab
```

# Lab 2.1 — PostgreSQL dynamic secrets

**Objective:** have Vault create and drop PostgreSQL users on demand, rotate an existing user's password, and lock away Vault's own database credential. **Time:** 35 min. **Study guide:** Day 2, Module 1. **Environment:** a fresh dev server + Docker. Keep this dev server running through Labs 2.3, 2.4 and 2.7 — they reuse what you build here.

## Steps

**Step 1 — Start a fresh dev server.**

```
$ vault server -dev -dev-root-token-id=root                    # terminal 1
$ export VAULT_ADDR=http://127.0.0.1:8200 VAULT_TOKEN=root     # terminal 2
```

**Step 2 — Start PostgreSQL and prepare it.** `vaultadmin` is *your* admin login; `vaultsvc` is the account Vault will use (and later rotate); `report_user` is a legacy account for the static-role step.

```
$ docker run -d --name pg -p 5432:5432 \
    -e POSTGRES_USER=vaultadmin -e POSTGRES_PASSWORD=VaultAdm1n -e POSTGRES_DB=appdb \
    postgres:16
$ sleep 5
$ PGPASSWORD=VaultAdm1n psql -h 127.0.0.1 -U vaultadmin -d appdb <<'SQL'
CREATE TABLE orders(id serial PRIMARY KEY, item text);
INSERT INTO orders(item) VALUES ('laptop'), ('phone');
CREATE ROLE vaultsvc WITH LOGIN SUPERUSER PASSWORD 'VaultSvc1n';
CREATE ROLE report_user LOGIN PASSWORD 'initial';
GRANT SELECT ON orders TO report_user;
SQL
```

`SUPERUSER` keeps the lab simple. In production give Vault's account `CREATEROLE` plus `GRANT ... WITH GRANT OPTION` on the schemas it manages.

**Step 3 — Create the three SQL files.**

```
-- app-ro.sql : creation statements for read-only roles
CREATE ROLE "{{name}}" WITH LOGIN PASSWORD '{{password}}' VALID UNTIL '{{expiration}}';
GRANT SELECT ON ALL TABLES IN SCHEMA public TO "{{name}}";
```

```
-- app-rw.sql : creation statements for the read-write role
CREATE ROLE "{{name}}" WITH LOGIN PASSWORD '{{password}}' VALID UNTIL '{{expiration}}';
GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA public TO "{{name}}";
GRANT USAGE, SELECT ON ALL SEQUENCES IN SCHEMA public TO "{{name}}";
```

```
-- revoke.sql : revocation statements shared by all roles
REVOKE ALL ON ALL TABLES IN SCHEMA public FROM "{{name}}";
REVOKE ALL ON ALL SEQUENCES IN SCHEMA public FROM "{{name}}";
DROP ROLE IF EXISTS "{{name}}";
```

**Step 4 — Enable the engine, register the connection, define roles.** `payments-ro` is used later in Labs 2.3 and 2.4.

```
$ vault secrets enable database
$ vault write database/config/appdb \
    plugin_name=postgresql-database-plugin \
```

```
    connection_url="postgresql://{{username}}:{{password}}@127.0.0.1:5432/appdb?
sslmode=disable" \
    username="vaultsvc" password="VaultSvc1n" \
    allowed_roles="app-ro,app-rw,payments-ro,legacy-report"
$ vault write database/roles/app-ro db_name=appdb \
    creation_statements=@app-ro.sql revocation_statements=@revoke.sql \
    default_ttl=1h max_ttl=24h
$ vault write database/roles/app-rw db_name=appdb \
    creation_statements=@app-rw.sql revocation_statements=@revoke.sql \
    default_ttl=30m max_ttl=4h
$ vault write database/roles/payments-ro db_name=appdb \
    creation_statements=@app-ro.sql revocation_statements=@revoke.sql \
    default_ttl=15m max_ttl=1h
```

Lab 2.1 · Steps 3-4 · Connection and roles — EXPECTED OUTPUT

```
trainee@vault-lab:~$ vault secrets enable database
Success! Enabled the database secrets engine at: database/
trainee@vault-lab:~$ vault write database/config/appdb plugin_name=postgresql-database-plugin co
                     nnection_url="postgresql://{{username}}:{{password}}@127.0.0.1:5432/app
                     db?sslmode=disable" username="vaultsvc" password="VaultSvc1n"
                     allowed_roles="app-ro,app-rw,payments-ro,legacy-report"
Success! Data written to: database/config/appdb
trainee@vault-lab:~$ vault write database/roles/app-ro db_name=appdb
                     creation_statements=@app-ro.sql revocation_statements=@revoke.sql
                     default_ttl=1h max_ttl=24h
Success! Data written to: database/roles/app-ro
trainee@vault-lab:~$ vault write database/roles/app-rw db_name=appdb
                     creation_statements=@app-rw.sql revocation_statements=@revoke.sql
                     default_ttl=30m max_ttl=4h
Success! Data written to: database/roles/app-rw
trainee@vault-lab:~$ vault write database/roles/payments-ro db_name=appdb
                     creation_statements=@app-ro.sql revocation_statements=@revoke.sql
                     default_ttl=15m max_ttl=1h
Success! Data written to: database/roles/payments-ro
```

*Expected output — engine enabled, connection and three roles written*

### Step 5 — Request credentials twice.

```
$ vault read database/creds/app-ro
$ vault read -field=username database/creds/app-ro
```

Lab 2.1 · Step 5 · Every read creates a new database user — EXPECTED OUTPUT

```
trainee@vault-lab:~$ vault read database/creds/app-ro
Key                Value
---                -----
lease_id           database/creds/app-ro/Xy7QpL2vN8mT4kR1wJ9cA3hD
lease_duration     1h
lease_renewable    true
password           A1a-8HqZt3mVx9pLw2Kc
username           v-root-app-ro-k2JdN8pQx4mT1vW7rL3c-1792150003
trainee@vault-lab:~$ vault read -field=username database/creds/app-ro
v-root-app-ro-Pq9mT2vN7xL4kW1rJ8cY-1792150011
```

*Expected output — two reads, two different users, each with a lease*

### Step 6 — Use the first credential, and look inside PostgreSQL.

```
$ PGPASSWORD='<password>' psql -h 127.0.0.1 -U '<username>' -d appdb -c "SELECT * FROM
orders;"
$ PGPASSWORD='<password>' psql -h 127.0.0.1 -U '<username>' -d appdb -c "INSERT INTO
orders(item) VALUES ('tablet');"
$ PGPASSWORD=VaultAdm1n psql -h 127.0.0.1 -U vaultadmin -d appdb \
    -c "SELECT usename, valuntil FROM pg_user WHERE usename LIKE 'v-%';"
```

Lab 2.1 · Step 6 · Use the credential; see it inside PostgreSQL — EXPECTED OUTPUT

```
trainee@vault-lab:~$ PGPASSWORD='A1a-8HqZt3mVx9pLw2Kc' psql -h 127.0.0.1 -U
                     'v-root-app-ro-k2JdN8pQx4mT1vW7rL3c-1792150003' -d appdb -c "SELECT *
                     FROM orders;"
 id |  item
----+--------
  1 | laptop
  2 | phone
(2 rows)

trainee@vault-lab:~$ PGPASSWORD='A1a-8HqZt3mVx9pLw2Kc' psql -h 127.0.0.1 -U
                     'v-root-app-ro-k2JdN8pQx4mT1vW7rL3c-1792150003' -d appdb -c "INSERT
                     INTO orders(item) VALUES ('tablet');"
ERROR:  permission denied for table orders
trainee@vault-lab:~$ PGPASSWORD=VaultAdm1n psql -h 127.0.0.1 -U vaultadmin -d appdb -c "SELECT
                     usename, valuntil FROM pg_user WHERE usename LIKE 'v-%';"
                   usename                   |        valuntil
---------------------------------------------+------------------------
 v-root-app-ro-k2JdN8pQx4mT1vW7rL3c-1792150003 | 2026-10-16 12:26:43+00
 v-root-app-ro-Pq9mT2vN7xL4kW1rJ8cY-1792150011 | 2026-10-16 12:26:51+00
(2 rows)
```

*Expected output — SELECT works, INSERT is denied, two v- users with expiry times*

### Step 7 — Revoke the first lease.

```
$ vault lease revoke <lease_id-from-step-5>
$ PGPASSWORD=VaultAdm1n psql -h 127.0.0.1 -U vaultadmin -d appdb \
    -c "SELECT usename FROM pg_user WHERE usename LIKE 'v-%';"
```

Lab 2.1 · Step 7 · Revoke the lease: the user is dropped — EXPECTED OUTPUT

```
trainee@vault-lab:~$ vault lease revoke database/creds/app-ro/Xy7QpL2vN8mT4kR1wJ9cA3hD
All revocation operations queued successfully!
trainee@vault-lab:~$ PGPASSWORD=VaultAdm1n psql -h 127.0.0.1 -U vaultadmin -d appdb -c "SELECT
                     usename FROM pg_user WHERE usename LIKE 'v-%';"
                   usename
---------------------------------------------
 v-root-app-ro-Pq9mT2vN7xL4kW1rJ8cY-1792150011
(1 row)
```

*Expected output — the revoked user is dropped; one remains*

### Steps 8–9 — Static role for the legacy account, then rotate Vault's own password.

```
$ vault write database/static-roles/legacy-report db_name=appdb \
    username=report_user rotation_period=24h
```

```
$ vault read database/static-creds/legacy-report
$ vault write -f database/rotate-role/legacy-report        # rotate now; read again to see a
new password

$ vault write -f database/rotate-root/appdb
$ PGPASSWORD=VaultSvc1n psql -h 127.0.0.1 -U vaultsvc -d appdb -c "SELECT 1;"
```

Lab 2.1 · Steps 8-9 · Static role and root rotation — EXPECTED OUTPUT

```
trainee@vault-lab:~$ vault write database/static-roles/legacy-report db_name=appdb
                       username=report_user rotation_period=24h
Success! Data written to: database/static-roles/legacy-report
trainee@vault-lab:~$ vault read database/static-creds/legacy-report
Key                    Value
---                    -----
last_vault_rotation    2026-10-16T11:31:05.214539Z
password               Zq4-xT8mPv2LnW7kR1cJ
rotation_period        24h
ttl                    23h59m52s
username               report_user
trainee@vault-lab:~$ vault write -f database/rotate-root/appdb
Success! Data written to: database/rotate-root/appdb
trainee@vault-lab:~$ PGPASSWORD=VaultSvc1n psql -h 127.0.0.1 -U vaultsvc -d appdb -c "SELECT 1;"
psql: error: connection to server at "127.0.0.1", port 5432 failed: FATAL:  password
    authentication failed for user "vaultsvc"
Only Vault knows vaultsvc's password now; vaultadmin (yours) still works.
```

*Expected output — static credentials for report_user; vaultsvc's old password no longer works*

✓ **Checkpoint:** `vault read database/creds/app-rw` still works after `rotate-root` — Vault knows the new password; you don't.

**Step 10 — The same flow from Python.**

```
# db_dynamic.py — get a fresh PostgreSQL user, use it, then hand it back
import os
import hvac
import psycopg2

client = hvac.Client(url=os.environ["VAULT_ADDR"], token=os.environ["VAULT_TOKEN"])

creds = client.secrets.database.generate_credentials(name="app-ro")
user, password = creds["data"]["username"], creds["data"]["password"]
lease_id, ttl = creds["lease_id"], creds["lease_duration"]
print(f"Got {user} for {ttl}s (lease {lease_id})")

with psycopg2.connect(host="127.0.0.1", dbname="appdb", user=user, password=password) as
conn:
    with conn.cursor() as cur:
        cur.execute("SELECT id, item FROM orders")
        print(cur.fetchall())

client.sys.revoke_lease(lease_id=lease_id)    # hand it back immediately
```

```
$ python3 db_dynamic.py
```

Lab 2.1 · Step 10 · The same flow from Python — EXPECTED OUTPUT

```
trainee@vault-lab:~$ python3 db_dynamic.py
Got v-root-app-ro-Wn4kT9vQ2mL7pX1rJ8cY-1792150644 for 3600s (lease
    database/creds/app-ro/Lm3qT8vN1pW4xK7rJ9cY2hA6)
[(1, 'laptop'), (2, 'phone')]
```

*Expected output — a new user and the two orders*

## Clean up

Keep the dev server and the `pg` container running for Labs 2.3, 2.4 and 2.7. Full clean-up is in the Appendix.

# Lab 2.2 — AWS and Azure dynamic secrets (optional)

**Objective:** issue short-lived cloud credentials from Vault. **Time:** 20 min. **Study guide:** Day 2, Module 1. **Environment:** dev server + a sandbox AWS account and/or Azure subscription. Do Part A, Part B, or both; the instructor can demo whichever cloud the batch uses.

## Part A — AWS

**Prerequisites:** an IAM user for Vault with the policy from the study guide (Day 2, Module 1), its access key, and an IAM role `vault-deploy` whose trust policy allows that IAM user to assume it.

**Step A1 — Configure the engine and two roles.**

```
$ vault secrets enable aws
$ vault write aws/config/root \
    access_key=<vault-iam-user-access-key> secret_key=<vault-iam-user-secret-key> region=ap-
south-1

$ vault write aws/roles/s3-readonly credential_type=iam_user \
    policy_arns=arn:aws:iam::aws:policy/AmazonS3ReadOnlyAccess
$ vault write aws/roles/deploy credential_type=assumed_role \
    role_arns=arn:aws:iam::<account-id>:role/vault-deploy \
    default_sts_ttl=1h max_sts_ttl=4h
```

**Step A2 — Request both kinds of credentials and test the STS one.**

```
$ vault read aws/creds/s3-readonly
$ vault write aws/sts/deploy ttl=30m
$ export AWS_ACCESS_KEY_ID=<access_key> AWS_SECRET_ACCESS_KEY=<secret_key> AWS_SESSION_TOKEN=
<session_token>
$ aws sts get-caller-identity
```

Lab 2.2 · Part A · AWS: IAM user and STS credentials — EXPECTED OUTPUT

```
trainee@vault-lab:~$ vault read aws/creds/s3-readonly
Key                Value
---                -----
lease_id           aws/creds/s3-readonly/9nKq2mT7vW4pL1xR8cJ3yA5h
lease_duration     768h
lease_renewable    true
access_key         AKIAEXAMPLE7QZ3W4N5V
secret_key         wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLE01
session_token      <nil>
trainee@vault-lab:~$ vault write aws/sts/deploy ttl=30m
Key                Value
---                -----
lease_id           aws/sts/deploy/Tq1mV8nW3pL6xK9rJ2cY5hA4
lease_duration     29m59s
lease_renewable    false
access_key         ASIAEXAMPLE2LK8MQ4R7
secret_key         Xp9qR2vT7mN4kL1wJ8cY3hA6sD0fG5eB2uZoKiVb
session_token      IQoJb3JpZ2luX2VjEXAMPLE...
trainee@vault-lab:~$ aws sts get-caller-identity
{
    "UserId": "AROAEXAMPLEROLEID:vault-root-deploy-1792151204-8kQ2",
    "Account": "123456789012",
    "Arn":
    "arn:aws:sts::123456789012:assumed-role/vault-deploy/vault-root-deploy-1792151204-8kQ2"
}
```

*Expected output — an IAM user key, then STS credentials for the assumed role*

✓ **Checkpoint:** the caller identity is an `assumed-role/vault-deploy/...` ARN. In the IAM console, the `iam_user` credential appears as a user named `vault-...`; `vault lease revoke -prefix aws/creds/s3-readonly` deletes it.

New IAM keys can take a few seconds to work. If the first call fails with `InvalidClientTokenId`, wait and retry.

**Step A3 (optional) — Rotate Vault's own key:** `vault write -f aws/config/rotate-root`. Afterwards only Vault knows the IAM user's secret key.

## Part B — Azure

**Prerequisites:** a service principal for Vault with Microsoft Graph `Application.ReadWrite.OwnedBy` (admin-consented) and Owner or User Access Administrator on resource group `rg-training`.

**Step B1 — Configure the engine and a role.**

```
$ vault secrets enable azure
$ vault write azure/config \
    subscription_id=<subscription-id> tenant_id=<tenant-id> \
    client_id=<vault-sp-app-id> client_secret=<vault-sp-secret>

$ vault write azure/roles/rg-contributor ttl=1h max_ttl=4h azure_roles=-<<EOF
[
  {
    "role_name": "Contributor",
    "scope": "/subscriptions/<subscription-id>/resourceGroups/rg-training"
  }
```

```
]
EOF
```

**Step B2 — Request a service principal and use it.**

```
$ vault read azure/creds/rg-contributor
$ az login --service-principal -u <client_id> -p '<client_secret>' --tenant <tenant-id> -o
none
$ az group show -n rg-training --query "{name:name, location:location}" -o table
```

Lab 2.2 · Part B · Azure: a short-lived service principal — EXPECTED OUTPUT

```
trainee@vault-lab:~$ vault read azure/creds/rg-contributor
Key                Value
---                -----
lease_id           azure/creds/rg-contributor/Hn7qT2vM9pW4kL1xR8cJ3yA5
lease_duration     1h
lease_renewable    true
client_id          3f1c9a7e-5b2d-4e8a-9c21-7d4e5f6a8b90
client_secret      Qx8~vN2mT7pL4kW1rJ9cY3hA6sD0fG5eB
trainee@vault-lab:~$ az login --service-principal -u 3f1c9a7e-5b2d-4e8a-9c21-7d4e5f6a8b90 -p
                     'Qx8~vN2mT7pL4kW1rJ9cY3hA6sD0fG5eB' --tenant <tenant-id> -o none
trainee@vault-lab:~$ az group show -n rg-training --query "{name:name, location:location}" -o
                     table
Name         Location
-----------  ------------
rg-training  centralindia
```

*Expected output — a client_id/secret pair that can read rg-training*

Role assignments can take a minute to propagate; retry `az group show` if it returns `AuthorizationFailed`.

**Step B3 — Revoke and verify.** `vault lease revoke -prefix azure/creds/rg-contributor`, then repeat `az group show`: the service principal is gone from Entra ID, and a fresh az login with the same client_id and secret now fails (an existing az session may keep working until its access token expires).

## Clean up

```
$ vault lease revoke -prefix aws/ ; vault secrets disable aws
$ vault lease revoke -prefix azure/ ; vault secrets disable azure
```

# Lab 2.3 — Least-privilege policies and identity groups

**Objective:** write, test and debug a least-privilege policy; then grant policies through identity groups and templates instead of per-user. **Time:** 25 min. **Study guide:** Day 2, Module 2. **Environment:** the dev server from Lab 2.1 ( `VAULT_TOKEN=root` ).

## Steps

**Step 1 — Seed two secrets.**

```
$ vault kv put secret/payments/api key=pay-123
$ vault kv put secret/payments/signing-key pem=SECRET
```

**Step 2 — Write the application policy.**

```
# payments-app.hcl
path "secret/data/payments/*" {
  capabilities = ["read"]
}

path "secret/metadata/payments/*" {
  capabilities = ["list"]
}

path "database/creds/payments-ro" {
  capabilities = ["read"]
}

# Never allow this app to touch the payments signing key
path "secret/data/payments/signing-key" {
  capabilities = ["deny"]
}
```

**Steps 3–4 — Load it, mint a token, ask Vault what the token may do.**

```
$ vault policy fmt payments-app.hcl
$ vault policy write payments-app payments-app.hcl
$ T=$(vault token create -policy=payments-app -ttl=15m -field=token)
$ vault token capabilities $T secret/data/payments/api
$ vault token capabilities $T secret/data/payments/signing-key
$ vault token capabilities $T sys/policies/acl/payments-app
$ vault token capabilities $T database/creds/payments-ro
```

Lab 2.3 · Steps 2-4 · Ask Vault what a token may do — EXPECTED OUTPUT

```
trainee@vault-lab:~$ vault policy fmt payments-app.hcl
Success! Formatted policy: payments-app.hcl
trainee@vault-lab:~$ vault policy write payments-app payments-app.hcl
Success! Uploaded policy: payments-app
trainee@vault-lab:~$ T=$(vault token create -policy=payments-app -ttl=15m -field=token)
trainee@vault-lab:~$ vault token capabilities $T secret/data/payments/api
read
trainee@vault-lab:~$ vault token capabilities $T secret/data/payments/signing-key
deny
trainee@vault-lab:~$ vault token capabilities $T sys/policies/acl/payments-app
deny
trainee@vault-lab:~$ vault token capabilities $T database/creds/payments-ro
read
```

*Expected output — read, deny (explicit), deny (no rule), read*

**Step 5 — Prove it, inspect the effective ACL, and generate a policy.**

```
$ VAULT_TOKEN=$T vault kv get -field=key secret/payments/api
$ VAULT_TOKEN=$T vault kv get secret/payments/signing-key
$ VAULT_TOKEN=$T vault read -format=json sys/internal/ui/resultant-acl | \
    jq -c '.data.glob_paths | with_entries(select(.key | startswith("secret")))'
$ vault kv get -output-policy secret/payments/api
```

Lab 2.3 · Step 5 · Prove it, then generate a policy — EXPECTED OUTPUT

```
trainee@vault-lab:~$ VAULT_TOKEN=$T vault kv get -field=key secret/payments/api
pay-123
trainee@vault-lab:~$ VAULT_TOKEN=$T vault kv get secret/payments/signing-key
Error reading secret/data/payments/signing-key: Error making API request.

URL: GET http://127.0.0.1:8200/v1/secret/data/payments/signing-key
Code: 403. Errors:

* 1 error occurred:
	* permission denied
trainee@vault-lab:~$ VAULT_TOKEN=$T vault read -format=json sys/internal/ui/resultant-acl | jq
                       -c '.data.glob_paths | with_entries(select(.key |
                       startswith("secret")))'
{"secret/data/payments/":{"capabilities":["read"]},"secret/metadata/payments/":{"capabilities":[
    "list"]}}
trainee@vault-lab:~$ vault kv get -output-policy secret/payments/api
path "secret/data/payments/api" {
  capabilities = ["read"]
}
```

*Expected output — api readable, signing-key 403, resultant ACL, generated policy*

✓ **Checkpoint:** `VAULT_TOKEN=$T vault read database/creds/payments-ro` returns a PostgreSQL user (role from Lab 2.1), and `VAULT_TOKEN=$T vault policy list` returns 403.

**Step 6 — The specificity gotcha.** Predict the answer before you run the last line.

```
$ vault policy write gotcha - <<'EOF'
path "secret/data/app/*" {
  capabilities = ["create", "read", "update"]
```

```
}
path "secret/data/app/config" {
  capabilities = ["read"]
}
EOF
$ G=$(vault token create -policy=gotcha -field=token)
$ vault token capabilities $G secret/data/app/db        # create, read, update
$ vault token capabilities $G secret/data/app/config    # read only: the exact path wins
```

### Step 7 — Grant through an identity group.

```
$ vault auth enable userpass
$ vault write auth/userpass/users/alice password='Passw0rd!' token_policies=default
$ ENTITY=$(vault write -field=id identity/entity name=alice policies=default)
$ ACC=$(vault auth list -format=json | jq -r '.["userpass/"].accessor')
$ vault write identity/entity-alias name=alice canonical_id=$ENTITY mount_accessor=$ACC
$ vault write identity/group name=payments-team policies=payments-app
member_entity_ids=$ENTITY
$ vault login -method=userpass username=alice | grep policies      # password Passw0rd!
$ export VAULT_TOKEN=root
```

Lab 2.3 · Steps 6-7 · Policies through an identity group — EXPECTED OUTPUT

```
trainee@vault-lab:~$ ENTITY=$(vault write -field=id identity/entity name=alice policies=default)
trainee@vault-lab:~$ ACC=$(vault auth list -format=json | jq -r '.["userpass/"].accessor')
trainee@vault-lab:~$ vault write identity/entity-alias name=alice canonical_id=$ENTITY
                       mount_accessor=$ACC
Key             Value
---             -----
canonical_id    6f1e2d3c-4b5a-6978-8a9b-0c1d2e3f4a5b
id              2b8c7d6e-5f4a-3b2c-1d0e-9f8a7b6c5d4e
trainee@vault-lab:~$ vault write identity/group name=payments-team policies=payments-app
                       member_entity_ids=$ENTITY
Key     Value
---     -----
id      9d3a4b5c-6d7e-8f90-a1b2-c3d4e5f6a7b8
name    payments-team
trainee@vault-lab:~$ vault login -method=userpass username=alice | grep policies
token_policies         ["default"]
identity_policies      ["payments-app"]
policies               ["default" "payments-app"]
```

*Expected output — alice's token carries payments-app through the group*

### Step 8 — One templated policy for every user.

```
$ echo $ACC                     # e.g. auth_userpass_6a1b2c3d
$ cat > per-user.hcl <<EOF
path "secret/data/users/{{identity.entity.aliases.$ACC.name}}/*" {
  capabilities = ["create", "read", "update", "patch", "delete"]
}
path "secret/metadata/users/{{identity.entity.aliases.$ACC.name}}/*" {
  capabilities = ["list", "read", "delete"]
}
EOF
$ vault policy write per-user per-user.hcl
$ vault write identity/group name=payments-team policies=payments-app,per-user
member_entity_ids=$ENTITY
$ A=$(vault login -token-only -method=userpass username=alice password='Passw0rd!')
```

```
$ VAULT_TOKEN=$A vault kv put secret/users/alice/notes todo="rotate keys"     # allowed
$ VAULT_TOKEN=$A vault kv put secret/users/bob/notes x=y                       # 403
```

The heredoc is unquoted on purpose so the shell fills in `$ACC`; the `{{...}}` braces pass through unchanged.

✓ **Checkpoint:** alice can write only under `secret/users/alice/`, and nobody edited her user to make that happen.

**Step 9 (optional) — Parameter constraints.** Load the `helpdesk.hcl` policy from the study guide, mint a token with it, and confirm `vault write auth/userpass/users/carol password=x token_policies=dev-read` succeeds while `token_policies=admin` is rejected.

## Clean up

Keep everything: Lab 2.4 uses `payments-app`.

# Lab 2.4 — AppRole and Vault Agent

**Objective:** log a machine in with AppRole, force secure introduction with response wrapping, and let Vault Agent handle login and secret rendering. **Time:** 30 min. **Study guide:** Day 2, Module 3. **Environment:** the dev server from Labs 2.1–2.3 (needs `payments-app` and `database/roles/payments-ro`).

## Steps

### Steps 1–3 — Create the role and fetch both halves of the credential.

```
$ vault auth enable approle
$ vault write auth/approle/role/payments \
    token_policies=payments-app token_ttl=1h token_max_ttl=4h \
    secret_id_ttl=10m secret_id_num_uses=1
$ vault read -field=role_id auth/approle/role/payments/role-id
$ vault write -f auth/approle/role/payments/secret-id
```

Lab 2.4 · Steps 1-3 · Role, role_id and a secret_id — EXPECTED OUTPUT

```
trainee@vault-lab:~$ vault auth enable approle
Success! Enabled approle auth method at: approle/
trainee@vault-lab:~$ vault write auth/approle/role/payments token_policies=payments-app
                     token_ttl=1h token_max_ttl=4h secret_id_ttl=10m secret_id_num_uses=1
Success! Data written to: auth/approle/role/payments
trainee@vault-lab:~$ vault read -field=role_id auth/approle/role/payments/role-id
3c9a7e1f-5b2d-4e8a-9c21-7d4e5f6a8b90
trainee@vault-lab:~$ vault write -f auth/approle/role/payments/secret-id
Key                   Value
---                   -----
secret_id             8b2f6d4e-1a3c-4b5d-9e7f-0a1b2c3d4e5f
secret_id_accessor    e4d3c2b1-a0f9-4e8d-7c6b-5a4938271605
secret_id_num_uses    1
secret_id_ttl         10m
```

*Expected output — role_id, and a single-use secret_id valid for 10 minutes*

### Step 4 — Log in, then try the same secret_id again.

```
$ vault write auth/approle/login role_id=<role_id> secret_id=<secret_id>
$ vault write auth/approle/login role_id=<role_id> secret_id=<secret_id>
```

Lab 2.4 · Step 4 · Log in once; a reused secret_id is rejected — EXPECTED OUTPUT

```
trainee@vault-lab:~$ vault write auth/approle/login role_id=3c9a7e1f-5b2d-4e8a-9c21-7d4e5f6a8b90
                       secret_id=8b2f6d4e-1a3c-4b5d-9e7f-0a1b2c3d4e5f
Key                   Value
---                   -----
token                 hvs.CAESIFq8vN2mT7pL4kW1rJ9cY3hA
token_accessor        Rk2mT8vN1pW4xL7qJ9cY3hA6
token_duration        1h
token_renewable       true
token_policies        ["default" "payments-app"]
identity_policies     []
policies              ["default" "payments-app"]
token_meta_role_name  payments
trainee@vault-lab:~$ vault write auth/approle/login role_id=3c9a7e1f-5b2d-4e8a-9c21-7d4e5f6a8b90
                       secret_id=8b2f6d4e-1a3c-4b5d-9e7f-0a1b2c3d4e5f
Error writing data to auth/approle/login: Error making API request.

URL: PUT http://127.0.0.1:8200/v1/auth/approle/login
Code: 400. Errors:

* invalid secret id
```

*Expected output — first login returns a payments-app token; the reuse is rejected*

✓ **Checkpoint:** `secret_id_num_uses=1` made the second login fail with `invalid secret id`.

**Step 5 — Give the CI pipeline a policy that forces wrapping.**

```
# ci-payments.hcl
path "auth/approle/role/payments/secret-id" {
  capabilities     = ["update"]
  min_wrapping_ttl = "1s"
  max_wrapping_ttl = "300s"
}
```

```
$ vault policy write ci-payments ci-payments.hcl
```

**Step 6 — Act as CI: unwrapped is refused, wrapped works.**

```
$ CI=$(vault token create -policy=ci-payments -ttl=15m -field=token)
$ VAULT_TOKEN=$CI vault write -f auth/approle/role/payments/secret-id
$ VAULT_TOKEN=$CI vault write -wrap-ttl=120s -f auth/approle/role/payments/secret-id
```

Lab 2.4 · Steps 5-6 · The CI token can only get it wrapped — EXPECTED OUTPUT

```
trainee@vault-lab:~$ CI=$(vault token create -policy=ci-payments -ttl=15m -field=token)
trainee@vault-lab:~$ VAULT_TOKEN=$CI vault write -f auth/approle/role/payments/secret-id
Error writing data to auth/approle/role/payments/secret-id: Error making API request.
(rejected: the ci-payments policy requires a wrap TTL between 1s and 300s)
trainee@vault-lab:~$ VAULT_TOKEN=$CI vault write -wrap-ttl=120s -f
                     auth/approle/role/payments/secret-id
Key                              Value
---                              -----
wrapping_token:                  hvs.CAESIJ7qT2vM9pW4kL1xR8cJ3yA5
wrapping_accessor:               Lq2mT8vN1pW4xK7rJ9cY3hA6
wrapping_token_ttl:              2m
wrapping_token_creation_time:    2026-10-16 11:52:09.712044 +0000 UTC
wrapping_token_creation_path:    auth/approle/role/payments/secret-id
wrapped_accessor:                a7b6c5d4-e3f2-4a1b-9c8d-7e6f5a4b3c2d
```

*Expected output — the plain request fails; the wrapped one returns only a wrapping token*

✓ **Checkpoint:** `wrapping_token_creation_path` is `auth/approle/role/payments/secret-id`. CI never saw the secret_id value.

**Step 7 — Prepare Vault Agent.** Mint a fresh wrapped secret_id and unwrap it straight into the file Agent will read.

```
$ sudo mkdir -p /etc/vault /etc/app
$ vault read -field=role_id auth/approle/role/payments/role-id | sudo tee /etc/vault/role_id
>/dev/null
$ WRAP=$(VAULT_TOKEN=$CI vault write -wrap-ttl=120s -field=wrapping_token -f
auth/approle/role/payments/secret-id)
$ VAULT_TOKEN=$WRAP vault unwrap -field=secret_id | sudo tee /etc/vault/secret_id >/dev/null
```

```
# /etc/vault/agent.hcl
vault {
  address = "http://127.0.0.1:8200"
}

auto_auth {
  method "approle" {
    config = {
      role_id_file_path                   = "/etc/vault/role_id"
      secret_id_file_path                 = "/etc/vault/secret_id"
      remove_secret_id_file_after_reading = true
    }
  }
  sink "file" {
    config = { path = "/etc/vault/token" }
  }
}

template {
  source      = "/etc/vault/db.ctmpl"
  destination = "/etc/app/db.env"
}
```

```
{{/* /etc/vault/db.ctmpl */}}
{{ with secret "database/creds/payments-ro" }}
DB_USER={{ .Data.username }}
```

```
DB_PASS={{ .Data.password }}
{{ end }}
```

**Step 8 — Run Agent and read what it rendered.**

```
$ sudo vault agent -config=/etc/vault/agent.hcl          # terminal 3; leave running
$ sudo cat /etc/app/db.env                                # terminal 2
$ ls /etc/vault/                                          # secret_id file is gone
```

Lab 2.4 · Steps 7-8 · Vault Agent logs in and renders the file — EXPECTED OUTPUT

```
trainee@vault-lab:~$ sudo vault agent -config=/etc/vault/agent.hcl
==> Vault Agent started! Log data will stream in below:

[INFO]  agent.auth.handler: starting auth handler
[INFO]  agent.auth.handler: authenticating
[INFO]  agent.auth.handler: authentication successful, sending token to sinks
[INFO]  agent.sink.file: token written: path=/etc/vault/token
[INFO]  agent: (runner) rendered "/etc/vault/db.ctmpl" => "/etc/app/db.env"
--- second terminal ---
trainee@vault-lab:~$ sudo cat /etc/app/db.env

DB_USER=v-approle-payments-Mq2kT8vN1pW4xL7rJ9cY-1792151530
DB_PASS=Hk3-mQ7vT2xN9pL4wR8c
```

*Expected output — Agent authenticates, writes the token sink and renders db.env*

✓ **Checkpoint:** `db.env` holds a `v-approle-payments-...` user, and `/etc/vault/secret_id` was deleted after use. Leave Agent running for 15 minutes (the `payments-ro` TTL) and `cat` the file again: Agent re-renders it with a new user before the old one expires.

## Clean up

Press Ctrl+C on Agent, then `sudo rm -rf /etc/vault /etc/app`. Keep the dev server for Lab 2.7.

# Lab 2.5 — Kubernetes auth with kind and the Agent Injector

**Objective:** let a pod authenticate with its service account and receive a secret through the Vault Agent sidecar, with no secret in the pod spec. **Time:** 35 min. **Study guide:** Day 2, Module 3. **Environment:** Docker, `kind`, `kubectl`, `helm` (see One-time machine setup). This lab runs its own Vault inside the cluster, so your dev server can keep running.

## Steps

### Step 1 — Create a cluster.

```
$ kind create cluster --name vault-lab
```

### Step 2 — Install Vault (dev mode) and the injector with Helm.

```
$ helm repo add hashicorp https://helm.releases.hashicorp.com && helm repo update
$ helm install vault hashicorp/vault \
    --set server.dev.enabled=true --set server.dev.devRootToken=root \
    --set injector.enabled=true
$ kubectl wait --for=condition=Ready pod/vault-0 --timeout=120s
$ kubectl get pods
```

Lab 2.5 · Steps 1-2 · kind cluster and Vault via Helm — EXPECTED OUTPUT

```
trainee@vault-lab:~$ kind create cluster --name vault-lab
Creating cluster "vault-lab" ...
 ✓ Ensuring node image
 ✓ Preparing nodes
 ✓ Writing configuration
 ✓ Starting control-plane
 ✓ Installing CNI
 ✓ Installing StorageClass
Set kubectl context to "kind-vault-lab"
trainee@vault-lab:~$ helm install vault hashicorp/vault --set server.dev.enabled=true --set
                       server.dev.devRootToken=root --set injector.enabled=true
NAME: vault
LAST DEPLOYED: Fri Oct 16 12:02:18 2026
NAMESPACE: default
STATUS: deployed
REVISION: 1
trainee@vault-lab:~$ kubectl get pods
NAME                                    READY   STATUS    RESTARTS   AGE
vault-0                                 1/1     Running   0          48s
vault-agent-injector-6f8b9c7d5b-x2k4p   1/1     Running   0          48s
```

*Expected output — cluster created; vault-0 and the injector are Running*

### Step 3 — Configure Vault from inside its pod.

```
$ kubectl exec -it vault-0 -- sh
/ $ vault kv put secret/payments/api key=pay-123
/ $ cat > /tmp/payments-k8s.hcl <<'EOF'
path "secret/data/payments/*" {
  capabilities = ["read"]
}
EOF
```

```
/ $ vault policy write payments-k8s /tmp/payments-k8s.hcl
/ $ vault auth enable kubernetes
/ $ vault write auth/kubernetes/config
kubernetes_host="https://$KUBERNETES_PORT_443_TCP_ADDR:443"
/ $ vault write auth/kubernetes/role/payments \
      bound_service_account_names=payments-app \
      bound_service_account_namespaces=payments \
      token_policies=payments-k8s token_ttl=1h
/ $ exit
```

Lab 2.5 · Step 3 · Configure Kubernetes auth inside the Vault pod — EXPECTED OUTPUT

```
trainee@vault-lab:~$ kubectl exec -it vault-0 -- sh
/ $ vault kv put secret/payments/api key=pay-123
/ $ vault policy write payments-k8s /tmp/payments-k8s.hcl
Success! Uploaded policy: payments-k8s
/ $ vault auth enable kubernetes
Success! Enabled kubernetes auth method at: kubernetes/
/ $ vault write auth/kubernetes/config
        kubernetes_host="https://$KUBERNETES_PORT_443_TCP_ADDR:443"
Success! Data written to: auth/kubernetes/config
/ $ vault write auth/kubernetes/role/payments bound_service_account_names=payments-app
        bound_service_account_namespaces=payments token_policies=payments-k8s token_ttl=1h
Success! Data written to: auth/kubernetes/role/payments
/ $ exit
```

*Expected output — policy, auth method, config and role written*

Vault runs inside the cluster, so it reviews tokens with its own service account. A Vault outside the cluster also needs `token_reviewer_jwt` and `kubernetes_ca_cert` in the config (study guide, Day 2 Module 3).

**Step 4 — Create the namespace, service account and deployment.**

```
# payments-deploy.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: payments
  namespace: payments
spec:
  replicas: 1
  selector:
    matchLabels: { app: payments }
  template:
    metadata:
      labels: { app: payments }
      annotations:
        vault.hashicorp.com/agent-inject: "true"
        vault.hashicorp.com/role: "payments"
        vault.hashicorp.com/agent-inject-secret-api.env: "secret/data/payments/api"
        vault.hashicorp.com/agent-inject-template-api.env: |
          {{- with secret "secret/data/payments/api" -}}
          API_KEY={{ .Data.data.key }}
          {{- end }}
    spec:
      serviceAccountName: payments-app
      containers:
        - name: app
          image: busybox:1.36
```

```
          command: ["sh", "-c", "while true; do cat /vault/secrets/api.env; echo; sleep 30;
done"]
```

**Step 5 — Deploy and check.**

```
$ kubectl create namespace payments
$ kubectl -n payments create serviceaccount payments-app
$ kubectl apply -f payments-deploy.yaml
$ kubectl -n payments get pods
$ kubectl -n payments logs deploy/payments -c app --tail=1
```

Lab 2.5 · Steps 4-5 · The injector delivers the secret — EXPECTED OUTPUT

```
trainee@vault-lab:~$ kubectl create namespace payments
namespace/payments created
trainee@vault-lab:~$ kubectl -n payments create serviceaccount payments-app
serviceaccount/payments-app created
trainee@vault-lab:~$ kubectl apply -f payments-deploy.yaml
deployment.apps/payments created
trainee@vault-lab:~$ kubectl -n payments get pods
NAME                        READY   STATUS    RESTARTS   AGE
payments-7c9d8b6f4d-q8w2m   2/2     Running   0          31s
trainee@vault-lab:~$ kubectl -n payments logs deploy/payments -c app --tail=1
API_KEY=pay-123
```

*Expected output — 2/2 containers (app + vault-agent); the app prints API_KEY*

✓ **Checkpoint:** `READY 2/2` and the log shows `API_KEY=pay-123`.

**Step 6 — See the raw login a pod makes.**

```
$ kubectl -n payments run tmp --rm -it --image=curlimages/curl \
    --overrides='{"spec":{"serviceAccountName":"payments-app"}}' -- sh
/ $ JWT=$(cat /var/run/secrets/kubernetes.io/serviceaccount/token)
/ $ curl -s -X POST -d "{\"role\":\"payments\",\"jwt\":\"$JWT\"}" \
      http://vault.default.svc:8200/v1/auth/kubernetes/login
/ $ exit
```

Lab 2.5 · Step 6 · The raw login a pod makes — EXPECTED OUTPUT

```
trainee@vault-lab:~$ kubectl -n payments run tmp --rm -it --image=curlimages/curl
                     --overrides='{"spec":{"serviceAccountName":"payments-app"}}' -- sh
/ $ JWT=$(cat /var/run/secrets/kubernetes.io/serviceaccount/token)
/ $ curl -s -X POST -d "{\"role\":\"payments\",\"jwt\":\"$JWT\"}"
       http://vault.default.svc:8200/v1/auth/kubernetes/login
{"request_id":"4e1f7a2b-9c3d-4e5f-8a6b-7c8d9e0f1a2b","lease_id":"","renewable":false,"lease_dura
    tion":0,"data":null,"wrap_info":null,"warnings":null,"auth":{"client_token":"hvs.CAESIM...",
    "accessor":"Wq7...",
"policies":["default","payments-k8s"],"token_policies":["default","payments-k8s"],"metadata":{"r
    ole":"payments",
"service_account_name":"payments-app","service_account_namespace":"payments","service_account_ui
    d":"0c1d2e3f-..."},
"lease_duration":3600,"renewable":true,"entity_id":"...","token_type":"service","orphan":true}}
```

*Expected output — a Vault token with payments-k8s and the service account in its metadata*

**Step 7 — Negative test.** Repeat Step 6 without `--overrides` (the pod then uses the `default` service account). The login is refused because `default` is not in `bound_service_account_names`.

**Step 8 — Rotate the secret.** `kubectl exec vault-0 -- vault kv put secret/payments/api key=pay-456`, wait up to five minutes, and watch the app log pick up the new value as the sidecar re-renders.

## Clean up

```
$ kind delete cluster --name vault-lab
```

# Lab 2.6 — JWT for GitHub Actions and OIDC login (optional)

**Objective:** let a CI job and an engineer log in without any stored Vault credential. **Time:** 25 min. **Study guide:** Day 2, Module 3. **Environment:** Part A needs a Vault reachable from GitHub over HTTPS (for example a cloud VM, or the instructor's Vault) and a GitHub repository; Part B needs an Entra ID app registration. Use it as an instructor demo if trainees lack these.

## Part A — GitHub Actions with JWT

**Step A1 — Trust GitHub's token issuer and create a tightly bound role.** Replace the repository with yours.

```
$ vault auth enable jwt
$ vault write auth/jwt/config \
    oidc_discovery_url="https://token.actions.githubusercontent.com" \
    bound_issuer="https://token.actions.githubusercontent.com"

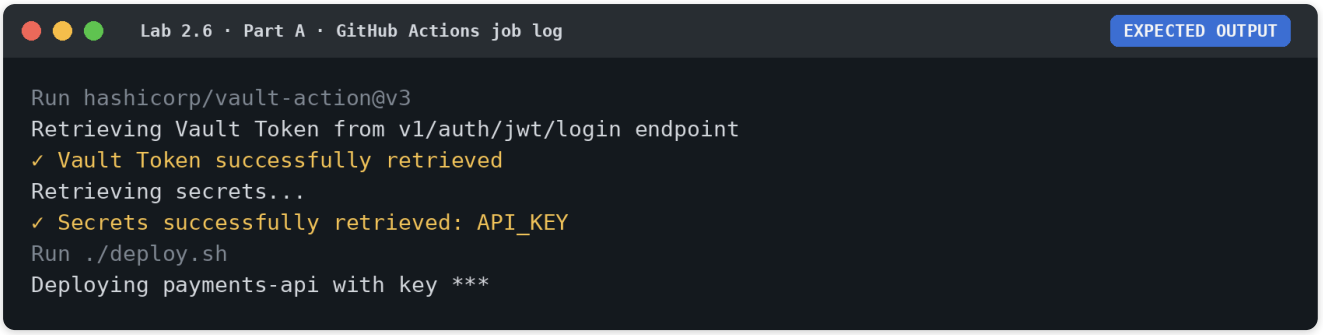
$ vault policy write payments-deployer - <<'EOF'
path "secret/data/payments/api" {
  capabilities = ["read"]
}
EOF

$ vault write auth/jwt/role/gha-deploy - <<'EOF'
{
  "role_type": "jwt",
  "user_claim": "actor",
  "bound_audiences": ["vault"],
  "bound_claims": {
    "repository": "stackmaster/payments-api",
    "ref": "refs/heads/main"
  },
  "token_policies": ["payments-deployer"],
  "token_ttl": "10m"
}
EOF
```

**Step A2 — Add the workflow and push to `main`.**

```
# .github/workflows/deploy.yml
name: deploy
on:
  push:
    branches: [main]
permissions:
  id-token: write        # lets the job request an OIDC token
  contents: read
jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: hashicorp/vault-action@v3
        with:
          url: https://vault.example.com:8200
          method: jwt
          path: jwt
          role: gha-deploy
          jwtGithubAudience: vault
```

```
          secrets: |
            secret/data/payments/api key | API_KEY
      - run: echo "Deploying payments-api with key $API_KEY"   # GitHub masks it as ***
```




*Expected output — the job obtains a Vault token and the secret, masked in the log*

**Step A3 — Negative test.** Push the same workflow on another branch (change `branches` to include it). The login fails because `ref` no longer matches `bound_claims`.

## Part B — OIDC for engineers (Microsoft Entra ID)

**Prerequisites:** an Entra ID app registration with redirect URIs `http://localhost:8250/oidc/callback` and `https://<vault-host>:8200/ui/vault/auth/oidc/oidc/callback`, a client secret, and the groups claim enabled in Token configuration.

**Step B1 — Configure the method and role.**

```
$ vault auth enable oidc
$ vault write auth/oidc/config \
    oidc_discovery_url="https://login.microsoftonline.com/<tenant-id>/v2.0" \
    oidc_client_id="<app-client-id>" oidc_client_secret="<client-secret>" \
    default_role=engineer
$ vault write auth/oidc/role/engineer \
    user_claim=preferred_username groups_claim=groups \
    oidc_scopes="openid,profile,email" token_policies=default token_ttl=8h \
    allowed_redirect_uris="http://localhost:8250/oidc/callback" \
    allowed_redirect_uris="https://<vault-host>:8200/ui/vault/auth/oidc/oidc/callback"
```

**Step B2 — Map an Entra group to a policy.**

```
$ vault policy write platform-admin admin.hcl          # admin.hcl from Lab 1.3
$ GID=$(vault write -field=id identity/group name=platform-engineers type=external
policies=platform-admin)
$ ACC=$(vault auth list -format=json | jq -r '.["oidc/"].accessor')
$ vault write identity/group-alias name=<entra-group-object-id> mount_accessor=$ACC
canonical_id=$GID
```

**Step B3 — Log in.**

```
$ vault login -method=oidc role=engineer
```

Lab 2.6 · Part B · OIDC login for an engineer — EXPECTED OUTPUT

```
trainee@vault-lab:~$ vault login -method=oidc role=engineer
Complete the login via your OIDC provider. Launching browser to:

    https://login.microsoftonline.com/<tenant-id>/oauth2/v2.0/authorize?client_id=...&redirect_u
    ri=http%3A%2F%2Flocalhost%3A8250%2Foidc%2Fcallback&response_type=code&scope=openid+profile+e
    mail

Waiting for OIDC authentication to complete...
Success! You are now authenticated. The token information displayed below
is already stored in the token helper.

Key                 Value
---                 -----
token_duration      8h
token_policies      ["default"]
identity_policies   ["platform-admin"]
policies            ["default" "platform-admin"]
token_meta_role     engineer
```

*Expected output — browser sign-in, then a token whose policies come from the Entra group*

✓ **Checkpoint:** `identity_policies` lists `platform-admin` even though the role itself grants only `default`.

## Clean up

`vault auth disable jwt; vault auth disable oidc`, and delete the test workflow.

# Lab 2.7 — Leases, renewal and revocation

**Objective:** inspect, renew and revoke leases one at a time and in bulk, and see that revoking a token revokes everything it obtained. **Time:** 20 min. **Study guide:** Day 2, Module 4. **Environment:** the dev server and `pg` container from Lab 2.1.

## Steps

### Steps 1–2 — Create three leases and inspect one.

```
$ for i in 1 2 3; do vault read -format=json database/creds/app-ro | jq -r .lease_id; done
$ vault list sys/leases/lookup/database/creds/app-ro
$ vault lease lookup database/creds/app-ro/<suffix>
```

Lab 2.7 · Steps 1-2 · Create and inspect leases — EXPECTED OUTPUT

```
trainee@vault-lab:~$ for i in 1 2 3; do vault read -format=json database/creds/app-ro | jq -r
                       .lease_id; done
database/creds/app-ro/2kTq8vN1pW4xL7rJ9cY3hA6s
database/creds/app-ro/Mq7vT2xN9pL4wR8cJ1kY5hA3
database/creds/app-ro/Zx1pW6qL9yJ2cH4vR8nT3mK7
trainee@vault-lab:~$ vault list sys/leases/lookup/database/creds/app-ro
Keys
----
2kTq8vN1pW4xL7rJ9cY3hA6s
Mq7vT2xN9pL4wR8cJ1kY5hA3
Zx1pW6qL9yJ2cH4vR8nT3mK7
trainee@vault-lab:~$ vault lease lookup database/creds/app-ro/2kTq8vN1pW4xL7rJ9cY3hA6s
Key             Value
---             -----
expire_time     2026-10-16T13:14:22.903817Z
id              database/creds/app-ro/2kTq8vN1pW4xL7rJ9cY3hA6s
issue_time      2026-10-16T12:14:22.903817Z
last_renewal    <nil>
renewable       true
ttl             59m48s
```

*Expected output — three lease IDs; one lease with about 60 minutes left*

### Step 3 — Renew.

```
$ vault lease renew database/creds/app-ro/<suffix>
$ vault lease renew -increment=2h database/creds/app-ro/<suffix>
```

Lab 2.7 · Step 3 · Renew — EXPECTED OUTPUT

```
trainee@vault-lab:~$ vault lease renew database/creds/app-ro/2kTq8vN1pW4xL7rJ9cY3hA6s
Key                Value
---                -----
lease_id           database/creds/app-ro/2kTq8vN1pW4xL7rJ9cY3hA6s
lease_duration     1h
lease_renewable    true
trainee@vault-lab:~$ vault lease renew -increment=2h
                       database/creds/app-ro/2kTq8vN1pW4xL7rJ9cY3hA6s
Key                Value
---                -----
lease_id           database/creds/app-ro/2kTq8vN1pW4xL7rJ9cY3hA6s
lease_duration     2h
lease_renewable    true
```

*Expected output — renewal resets the TTL; -increment asks for more*

**Step 4 — Hit the max TTL cap.** Tune the mount down, create a new lease, and ask for more than the cap allows.

```
$ vault secrets tune -default-lease-ttl=30m -max-lease-ttl=45m database/
$ L=$(vault read -format=json database/creds/app-ro | jq -r .lease_id)
$ vault lease renew -increment=2h $L | grep lease_duration     # granted about 45m, not 2h
$ vault secrets tune -default-lease-ttl=0 -max-lease-ttl=0 database/   # back to defaults
```

✓ **Checkpoint:** the renewal was granted far less than the 2h asked for — the mount's max TTL wins, exactly as in the study-guide lease timeline.

**Step 5 — Revoke one lease, then the whole role.**

```
$ vault lease revoke database/creds/app-ro/<suffix>
$ vault lease revoke -prefix database/creds/app-ro
$ PGPASSWORD=VaultAdm1n psql -h 127.0.0.1 -U vaultadmin -d appdb \
    -c "SELECT usename FROM pg_user WHERE usename LIKE 'v-%';"
```

Lab 2.7 · Steps 4-5 · Revoke one, then the whole role — EXPECTED OUTPUT

```
trainee@vault-lab:~$ vault lease revoke database/creds/app-ro/2kTq8vN1pW4xL7rJ9cY3hA6s
All revocation operations queued successfully!
trainee@vault-lab:~$ vault lease revoke -prefix database/creds/app-ro
All revocation operations queued successfully!
trainee@vault-lab:~$ PGPASSWORD=VaultAdm1n psql -h 127.0.0.1 -U vaultadmin -d appdb -c "SELECT
                       usename FROM pg_user WHERE usename LIKE 'v-%';"
 usename
---------
(0 rows)
```

*Expected output — revoke-prefix leaves no v- users in PostgreSQL*

**Step 6 — Revoking a token revokes its leases.**

```
$ vault policy write db-ro - <<'EOF'
path "database/creds/app-ro" {
  capabilities = ["read"]
}
EOF
```

```
$ T=$(vault token create -policy=db-ro -ttl=30m -field=token)
$ VAULT_TOKEN=$T vault read -field=username database/creds/app-ro
$ PGPASSWORD=VaultAdm1n psql -h 127.0.0.1 -U vaultadmin -d appdb \
    -tc "SELECT count(*) FROM pg_user WHERE usename LIKE 'v-%';"
$ vault token revoke $T
$ PGPASSWORD=VaultAdm1n psql -h 127.0.0.1 -U vaultadmin -d appdb \
    -tc "SELECT count(*) FROM pg_user WHERE usename LIKE 'v-%';"
```

Lab 2.7 · Step 6 · Revoking a token revokes its leases — EXPECTED OUTPUT

```
trainee@vault-lab:~$ T=$(vault token create -policy=db-ro -ttl=30m -field=token)
trainee@vault-lab:~$ VAULT_TOKEN=$T vault read -field=username database/creds/app-ro
v-token-app-ro-Hq3mT8vN1pW4xL7rJ9cY-1792153377
trainee@vault-lab:~$ PGPASSWORD=VaultAdm1n psql -h 127.0.0.1 -U vaultadmin -d appdb -tc "SELECT
                       count(*) FROM pg_user WHERE usename LIKE 'v-%';"
     1
trainee@vault-lab:~$ vault token revoke $T
Success! Revoked token (if it existed)
trainee@vault-lab:~$ PGPASSWORD=VaultAdm1n psql -h 127.0.0.1 -U vaultadmin -d appdb -tc "SELECT
                       count(*) FROM pg_user WHERE usename LIKE 'v-%';"
     0
```

*Expected output — one user while the token lives; zero after the token is revoked*

✓ **Checkpoint:** the count drops from 1 to 0 without touching the lease directly.

**Step 7 (optional) — Renew from code.** Run `lease_keeper.py` from the study guide (Day 2, Module 4) with `VAULT_TOKEN=root`; set the `app-ro` role's `default_ttl` to `2m` first (`vault write database/roles/app-ro default_ttl=2m max_ttl=6m creation_statements=@app-ro.sql revocation_statements=@revoke.sql db_name=appdb`) to watch renewals and a credential swap within minutes.

## Clean up

`vault lease revoke -prefix database/creds/` removes every remaining dynamic user. Keep the dev server for Lab 2.8.

# Lab 2.8 — Response wrapping and cubbyhole

**Objective:** prove that each token's cubbyhole is private, then deliver a secret through a single-use wrapping token and detect a replay. **Time:** 20 min. **Study guide:** Day 2, Module 5. **Environment:** the dev server ( `VAULT_TOKEN=root` ; `secret/payments/api` from Lab 2.3).

## Part A — Cubbyhole

```
$ vault write cubbyhole/notes msg="only my token can see this"
$ vault read cubbyhole/notes
$ T2=$(vault token create -policy=default -ttl=10m -field=token)
$ VAULT_TOKEN=$T2 vault read cubbyhole/notes
```

Lab 2.8 · Part A · Cubbyhole is private to each token — EXPECTED OUTPUT

```
trainee@vault-lab:~$ vault write cubbyhole/notes msg="only my token can see this"
Success! Data written to: cubbyhole/notes
trainee@vault-lab:~$ vault read cubbyhole/notes
Key    Value
---    -----
msg    only my token can see this
trainee@vault-lab:~$ T2=$(vault token create -policy=default -ttl=10m -field=token)
trainee@vault-lab:~$ VAULT_TOKEN=$T2 vault read cubbyhole/notes
No value found at cubbyhole/notes
```

*Expected output — root reads its note; T2 finds nothing at the same path*

Now write a note as T2, then `vault token revoke $T2` : its cubbyhole is destroyed with it. Even root cannot read another token's cubbyhole.

## Part B — Response wrapping

**Steps 1–2 — Wrap a read, then verify where it came from.** Lookup does not consume the token.

```
$ vault kv get -wrap-ttl=5m secret/payments/api
$ vault write sys/wrapping/lookup token=<wrapping_token>
```

Lab 2.8 · Part B · Steps 1-2 · Wrap and verify — EXPECTED OUTPUT

```
trainee@vault-lab:~$ vault kv get -wrap-ttl=5m secret/payments/api
Key                              Value
---                              -----
wrapping_token:                  hvs.CAESIOq4vT9mN2pL7kW1xR8cJ3yA
wrapping_accessor:               Kq3mT8vN1pW4xL7rJ9cY2hA6
wrapping_token_ttl:              5m
wrapping_token_creation_time:    2026-10-16 12:40:11.530718 +0000 UTC
wrapping_token_creation_path:    secret/data/payments/api
trainee@vault-lab:~$ vault write sys/wrapping/lookup token=hvs.CAESIOq4vT9mN2pL7kW1xR8cJ3yA
Key              Value
---              -----
creation_path    secret/data/payments/api
creation_time    2026-10-16T12:40:11.530718Z
creation_ttl     300
```

*Expected output — only a wrapping token is returned; lookup shows its creation_path*

**Steps 3–4 — Unwrap once; try again.**

```
$ vault unwrap <wrapping_token>
$ vault unwrap <wrapping_token>
```

Lab 2.8 · Part B · Steps 3-4 · Unwrap once; the second attempt fails — EXPECTED OUTPUT

```
trainee@vault-lab:~$ vault unwrap hvs.CAESIOq4vT9mN2pL7kW1xR8cJ3yA
Key         Value
---         -----
data        map[key:pay-123]
metadata    map[created_time:2026-10-16T12:21:03.118Z custom_metadata:<nil> deletion_time:
    destroyed:false version:1]
trainee@vault-lab:~$ vault unwrap hvs.CAESIOq4vT9mN2pL7kW1xR8cJ3yA
Error unwrapping: Error making API request.

URL: PUT http://127.0.0.1:8200/v1/sys/wrapping/unwrap
Code: 400. Errors:

* wrapping token is not valid or does not exist
```

*Expected output — the first unwrap returns the secret; the second fails*

✓ **Checkpoint:** the second unwrap fails with `wrapping token is not valid or does not exist`. In real life, that error on a token *you* never opened means someone else did.

**Step 5 — Simulate interception.** Work in pairs: trainee A wraps a secret and pastes the token in chat; trainee B unwraps it first; trainee A's unwrap now fails. Discuss what A should do (alert, revoke the source, re-issue).

**Step 6 — Wrap your own data, and extend a wrap without opening it.**

```
$ vault write -wrap-ttl=10m sys/wrapping/wrap db_pass="S3cret!" note="for vendor X"
$ vault write sys/wrapping/rewrap token=<wrapping_token>        # new token; the old one dies
```

**Step 7 — The same through the API and Python.**

```
$ curl -s -H "X-Vault-Token: $VAULT_TOKEN" -H "X-Vault-Wrap-TTL: 300" \
    $VAULT_ADDR/v1/secret/data/payments/api | jq .wrap_info
$ curl -s -X POST -H "X-Vault-Token: <wrapping_token>" $VAULT_ADDR/v1/sys/wrapping/unwrap |
jq .data.data
```

Then run `wrap_demo.py` from the study guide (Day 2, Module 5).

## Clean up

Nothing to remove; wrapping tokens expire on their own.

# Lab 2.9 — Day 2 capstone

**Objective:** a payments app starts with only a `role_id` and a wrapped `secret_id`; it verifies and unwraps the `secret_id`, logs in with AppRole, reads its config, gets a 15-minute PostgreSQL user, runs a query, and cleans up after itself. **Time:** 45 min. **Study guide:** Day 2, all modules. **Environment:** the dev server (root token) + Docker. It uses its own database on port 5433 and its own mount, so it does not disturb Labs 2.1–2.8.

| Script step | What happens | Module |
|---|---|---|
| 1 | PostgreSQL `payments` database with a `txns` table | 1 |
| 2 | Database engine at `db-payments/`, role `payments-ro` (TTL 15m, max 1h) | 1, 4 |
| 3 | Static app config in KV v2 | Day 1 |
| 4 | `payments-app` and `ci-payments` least-privilege policies | 2 |
| 5 | AppRole `payments`: single-use `secret_id`, 10-minute life | 3 |
| 6 | A CI token mints a `secret_id` it can only receive wrapped | 2, 3, 5 |
| App | Verify → unwrap → login → read → query → revoke | All |

## Step 1 — Create the setup script

```
#!/usr/bin/env bash
# day2-lab.sh — Day 2 end-to-end environment (LAB ONLY)
set -euo pipefail
export VAULT_ADDR="${VAULT_ADDR:-http://127.0.0.1:8200}"
: "${VAULT_TOKEN:?export VAULT_TOKEN as an admin or root token first}"
LAB="$HOME/vault-day2"; mkdir -p "$LAB"; cd "$LAB"

# 1. PostgreSQL on port 5433 (Lab 2.1 uses 5432)
docker rm -f pg-day2 >/dev/null 2>&1 || true
docker run -d --name pg-day2 -p 5433:5432 \
  -e POSTGRES_USER=vaultadmin -e POSTGRES_PASSWORD=VaultAdm1n -e POSTGRES_DB=payments \
  postgres:16 >/dev/null
export PGPASSWORD=VaultAdm1n
until psql -h 127.0.0.1 -p 5433 -U vaultadmin -d payments -c 'SELECT 1' >/dev/null 2>&1; do
sleep 1; done
psql -h 127.0.0.1 -p 5433 -U vaultadmin -d payments -q -c \
  "CREATE TABLE IF NOT EXISTS txns(id serial PRIMARY KEY, amount numeric, status text);
   INSERT INTO txns(amount, status) VALUES (499.00, 'settled'), (1250.50, 'pending');"
unset PGPASSWORD

# 2. Database secrets engine
vault secrets enable -path=db-payments database 2>/dev/null || true
vault write db-payments/config/payments plugin_name=postgresql-database-plugin \
  connection_url="postgresql://{{username}}:{{password}}@127.0.0.1:5433/payments?
sslmode=disable" \
  username=vaultadmin password=VaultAdm1n allowed_roles=payments-ro
vault write db-payments/roles/payments-ro db_name=payments default_ttl=15m max_ttl=1h \
  creation_statements="CREATE ROLE \"{{name}}\" WITH LOGIN PASSWORD '{{password}}' VALID
UNTIL '{{expiration}}'; GRANT SELECT ON ALL TABLES IN SCHEMA public TO \"{{name}}\";" \
  revocation_statements="REVOKE ALL ON ALL TABLES IN SCHEMA public FROM \"{{name}}\"; DROP
ROLE IF EXISTS \"{{name}}\";"

# 3. Static config
```

```
vault secrets enable -path=secret kv-v2 2>/dev/null || true
vault kv put secret/payments/config gateway_url=https://pay.lab.local merchant_id=SMT-001

# 4. Policies
vault policy write payments-app - <<'EOF'
path "db-payments/creds/payments-ro" {
  capabilities = ["read"]
}
path "secret/data/payments/config" {
  capabilities = ["read"]
}
EOF

vault policy write ci-payments - <<'EOF'
path "auth/approle/role/payments/secret-id" {
  capabilities     = ["update"]
  min_wrapping_ttl = "1s"
  max_wrapping_ttl = "300s"
}
EOF

# 5. AppRole
vault auth enable approle 2>/dev/null || true
vault write auth/approle/role/payments \
  token_policies=payments-app token_ttl=30m token_max_ttl=2h \
  secret_id_ttl=10m secret_id_num_uses=1
vault read -field=role_id auth/approle/role/payments/role-id > role_id
chmod 600 role_id

# 6. The CI pipeline, with its own narrow token, mints a WRAPPED secret_id
CI_TOKEN="$(vault token create -policy=ci-payments -ttl=15m -field=token)"
if VAULT_TOKEN="$CI_TOKEN" vault write -f auth/approle/role/payments/secret-id >/dev/null
2>&1; then
  echo "UNEXPECTED: CI got an unwrapped secret_id"
else
  echo "OK: policy forces wrapping"
fi
VAULT_TOKEN="$CI_TOKEN" vault write -f -wrap-ttl=120s -field=wrapping_token \
  auth/approle/role/payments/secret-id > wrapped_secret_id
chmod 600 wrapped_secret_id
echo "Ready: $LAB/role_id and $LAB/wrapped_secret_id (valid for 120s)"
```

This script overwrites the `payments-app` policy and the `payments` AppRole from Labs 2.3–2.4 with capstone versions.

### Step 2 — Run it

```
$ export VAULT_ADDR=http://127.0.0.1:8200 VAULT_TOKEN=root
$ chmod +x day2-lab.sh && ./day2-lab.sh
```

Lab 2.9 · Step 2 · Build the Day 2 environment — EXPECTED OUTPUT

```
trainee@vault-lab:~$ ./day2-lab.sh
Success! Enabled the database secrets engine at: db-payments/
Success! Data written to: db-payments/config/payments
Success! Data written to: db-payments/roles/payments-ro
==== Secret Path ====
secret/data/payments/config
(metadata omitted)
Success! Uploaded policy: payments-app
Success! Uploaded policy: ci-payments
Success! Data written to: auth/approle/role/payments
OK: policy forces wrapping
Ready: /home/trainee/vault-day2/role_id and /home/trainee/vault-day2/wrapped_secret_id (valid
    for 120s)
```

*Expected output — engine, roles and policies written; CI forced to wrap*

## Step 3 — Create the application

```
# app.py — secure introduction + dynamic DB credentials (pip install hvac psycopg2-binary)
import os
import sys

import hvac
import psycopg2

VAULT_ADDR = os.getenv("VAULT_ADDR", "http://127.0.0.1:8200")
LAB = os.path.expanduser("~/vault-day2")
EXPECTED_PATH = "auth/approle/role/payments/secret-id"

role_id = open(f"{LAB}/role_id").read().strip()
wrapping_token = open(f"{LAB}/wrapped_secret_id").read().strip()

# 1. Verify where the wrapped secret came from (lookup does not consume it)
try:
    info = hvac.Client(url=VAULT_ADDR).adapter.post(
        "/v1/sys/wrapping/lookup", json={"token": wrapping_token})
except hvac.exceptions.VaultError as e:
    sys.exit(f"ALERT: wrapping token invalid or already used: {e}")
if info["data"]["creation_path"] != EXPECTED_PATH:
    sys.exit(f"ALERT: unexpected creation_path {info['data']['creation_path']}")

# 2. Unwrap exactly once, using the wrapping token itself
secret_id = hvac.Client(url=VAULT_ADDR, token=wrapping_token).sys.unwrap()["data"]
["secret_id"]

# 3. AppRole login; the client keeps the resulting token
client = hvac.Client(url=VAULT_ADDR)
auth = client.auth.approle.login(role_id=role_id, secret_id=secret_id)["auth"]
print(f"Logged in: policies={auth['policies']} ttl={auth['lease_duration']}s")

# 4. Static config from KV v2
cfg = client.secrets.kv.v2.read_secret_version(
    mount_point="secret", path="payments/config", raise_on_deleted_version=True)["data"]
["data"]
print(f"Gateway {cfg['gateway_url']} merchant {cfg['merchant_id']}")

# 5. Dynamic, short-lived PostgreSQL user
creds = client.secrets.database.generate_credentials(name="payments-ro", mount_point="db-
```

```
payments")
user, password = creds["data"]["username"], creds["data"]["password"]
print(f"DB user {user}, lease {creds['lease_duration']}s")

with psycopg2.connect(host="127.0.0.1", port=5433, dbname="payments",
                      user=user, password=password) as conn:
    with conn.cursor() as cur:
        cur.execute("SELECT id, amount, status FROM txns ORDER BY id")
        for row in cur.fetchall():
            print("  txn", row)

# 6. Least privilege check: the app must not be able to write config
try:
    client.secrets.kv.v2.create_or_update_secret(
        mount_point="secret", path="payments/config", secret={"x": "y"})
    print("WARNING: write allowed — policy too broad")
except hvac.exceptions.Forbidden:
    print("Write denied, as expected")

# 7. Clean up: drop the DB user now and revoke our own token
client.sys.revoke_lease(lease_id=creds["lease_id"])
client.auth.token.revoke_self()
print("Lease and token revoked")
```

## Step 4 — Run the app within 120 seconds, then run it again

```
$ python3 app.py
$ python3 app.py          # same wrapped token: must refuse to start
```

Lab 2.9 · Steps 3-4 · Run the app, then run it again — EXPECTED OUTPUT

```
trainee@vault-lab:~$ python3 app.py
Logged in: policies=['default', 'payments-app'] ttl=1800s
Gateway https://pay.lab.local merchant SMT-001
DB user v-approle-payments-Tq8vN2mW4pL7kX1rJ9cY-1792154021, lease 900s
  txn (1, Decimal('499.00'), 'settled')
  txn (2, Decimal('1250.50'), 'pending')
Write denied, as expected
Lease and token revoked
trainee@vault-lab:~$ python3 app.py
ALERT: wrapping token invalid or already used: wrapping token is not valid or does not exist, on
     post http://127.0.0.1:8200/v1/sys/wrapping/lookup
```

*Expected output — full run, then the ALERT on reuse*

If the first run says the wrapping token is invalid, more than 120 seconds passed: re-run the script and start the app straight away — exactly what a deployment pipeline does on each release.

## Self-check

- ☐ The CI token cannot obtain an unwrapped `secret_id`
- ☐ The second `python3 app.py` prints the ALERT line
- ☐ While the app runs, PostgreSQL on port 5433 has one `v-approle-payments-...` user, and it is gone afterwards
- ☐ `vault list sys/leases/lookup/db-payments/creds/payments-ro` is empty after the run
- ☐ Changing `EXPECTED_PATH` makes the app refuse to start

**Stretch goals:** replace the Python login with Vault Agent (Lab 2.4); shorten `default_ttl` to 2 minutes and add the renewal loop from the study guide's Module 4; run the same app as a pod with Kubernetes auth (Lab 2.5).

## Clean up

```
$ docker rm -f pg-day2
$ vault lease revoke -prefix db-payments/ && vault secrets disable db-payments
$ rm -rf ~/vault-day2
```

# Appendix — Reset scripts and completion checklist

Run the reset that matches where you are; each one returns the machine to a clean starting point without touching the other environments.

## Reset scripts

```
# Dev server labs (1.2, 1.5–1.8, 2.1–2.4, 2.7–2.9): stop it and its data is gone
#   press Ctrl+C in the dev server terminal, then:
$ unset VAULT_TOKEN; rm -f ~/.vault-token

# Lab 1.3 systemd server: wipe and start over
$ sudo systemctl stop vault
$ sudo rm -rf /opt/vault/data/* /var/log/vault/audit.log
$ rm -f ~/vault-lab/init.txt

# Lab 1.9 scripted server
$ kill "$(cat ~/vault-lab/vault.pid)" 2>/dev/null; rm -rf ~/vault-lab

# Containers from Labs 1.7, 2.1 and 2.9
$ docker rm -f openldap pg pg-day2 vault-dev 2>/dev/null

# Lab 2.4 Vault Agent files
$ sudo rm -rf /etc/vault /etc/app

# Lab 2.5 cluster
$ kind delete cluster --name vault-lab
```

## Common lab problems

| Symptom | Fix |
|---|---|
| `bind: address already in use` on 8200 | Another Vault is running: `sudo systemctl stop vault`, Ctrl+C the dev server, or `kill $(cat ~/vault-lab/vault.pid)` |
| Every command returns 403 after a `vault login` | `VAULT_TOKEN` still points at a different token — `unset VAULT_TOKEN` or `export VAULT_TOKEN=root` deliberately |
| `server gave HTTP response to HTTPS client` | `VAULT_ADDR` must be `http://` for these labs |
| Lab 2.4/2.7 say a database role does not exist | The dev server was restarted and lost Lab 2.1 — redo Lab 2.1 Steps 1–4 |
| `psql: connection refused` right after `docker run` | PostgreSQL is still starting — wait 5 seconds |
| Wrapped `secret_id` invalid in Lab 2.9 | More than 120 s passed — re-run `day2-lab.sh` and start the app immediately |

## Completion checklist

- ☐ Lab 1.1 — three secrets found by grep; gitleaks run
- ☐ Lab 1.2 — dev server running; `secret/hello` written and read in the CLI and UI
- ☐ Lab 1.3 — Raft server initialised with 5/3, unsealed, audit on, root token revoked
- ☐ Lab 1.4 — Shamir demo prints True, True, False

- ☐ Lab 1.5 — same secret written through CLI, API and UI (3 versions)
- ☐ Lab 1.6 — `dev-read` token reads but cannot write; child token died with its parent; alice logged in
- ☐ Lab 1.7 — bob received `dev-read` through the LDAP `devops` group
- ☐ Lab 1.8 — KV v2 secret taken through delete, undelete, destroy, rollback and check-and-set
- ☐ Lab 1.9 — `day1-lab.sh` and `app.py` complete with "Write denied, as expected"
- ☐ Lab 2.1 — dynamic PostgreSQL users created and revoked; static role rotated; root rotated
- ☐ Lab 2.2 — (optional) AWS and/or Azure credentials issued and revoked
- ☐ Lab 2.3 — `payments-app` capabilities verified; alice gets policies via a group and a template
- ☐ Lab 2.4 — AppRole login works once per `secret_id`; CI forced to wrap; Agent rendered `db.env`
- ☐ Lab 2.5 — pod shows `API_KEY=pay-123` with no secret in its spec
- ☐ Lab 2.6 — (optional) GitHub Actions and/or OIDC login without stored credentials
- ☐ Lab 2.7 — lease renewed, capped, revoked by prefix, and revoked through its token
- ☐ Lab 2.8 — cubbyhole isolation shown; second unwrap failed
- ☐ Lab 2.9 — `app.py` ran once and refused the reused wrapping token

**Next:** Day 3 labs (3.1–3.10) and their reset scripts are in the Day 3 Lab Guide.